

Lecture 10

GPU and Advanced Shading

- The Graphics Processing Unit (GPU)
- Advanced Shading

The Graphics Processing Unit (GPU)

Latency

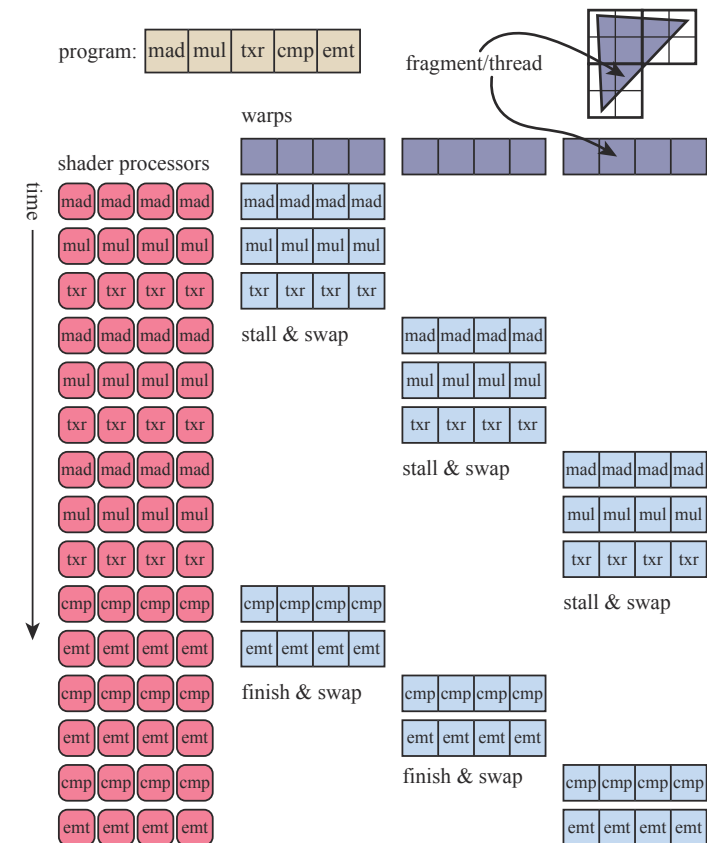
- All processors face latency
- Accessing data takes some amount of time
- The further away the information is from the processor, the longer the wait
- Local registers are faster than main memory
- Different architectures deal with latency in different ways

Latency

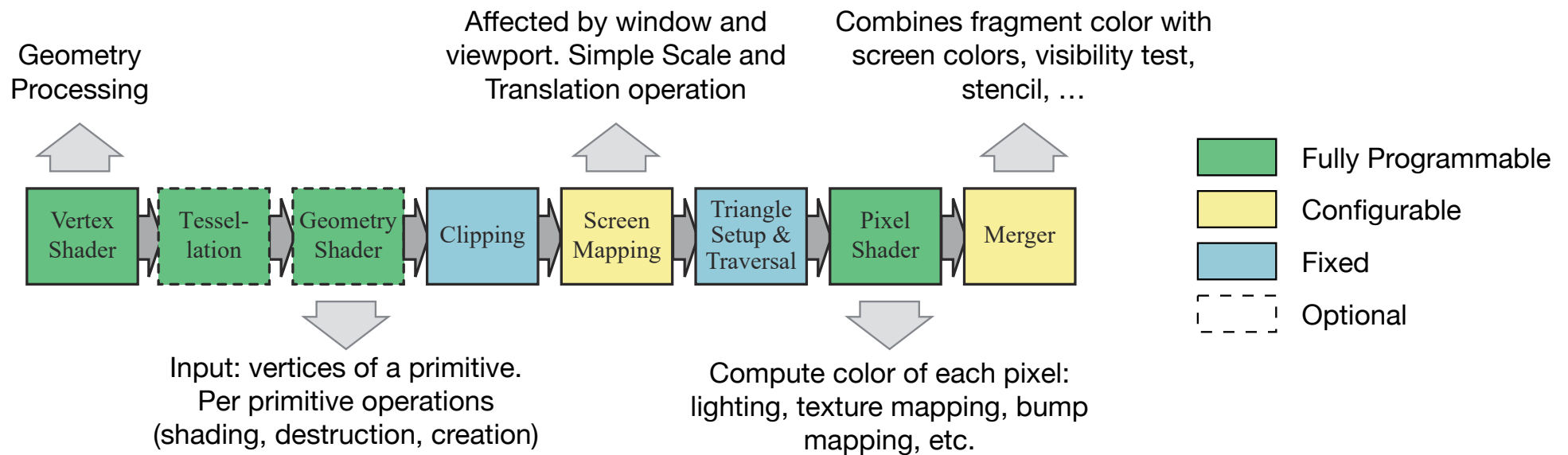
- CPUs have low core count and chip area can be occupied with large caches, circuits for speculative execution (branch prediction), instruction reordering, register renaming and cache pre-fetching.
- GPUs have a very large number of processor cores (in the thousands) working together as a very wide stream processor - ordered sets of similar data are processed in turn. Vertices or pixels are handled in a massively parallel fashion, with near to no interdependency, in a setting that aims for throughput. The higher core count and less sophisticated control logic leads to higher latency for each shader core.

Data-Parallel Architectures

- Threads are grouped in warps (wavefronts)
- Each thread has input memory and registers
- Warps execute the same instruction in lock-step mode (SIMD)
- When a thread/warp stalls, it is swapped out. No data needs to be copied!
- Threads that have started are named “in flight” (resident)
- The number of resident threads (occupancy) depends on the number of registers used by the shader program
- High occupancy promotes low latency
- Thread divergence (branching) reduces efficiency



GPU Pipeline Overview



The Programmable Shader Stage

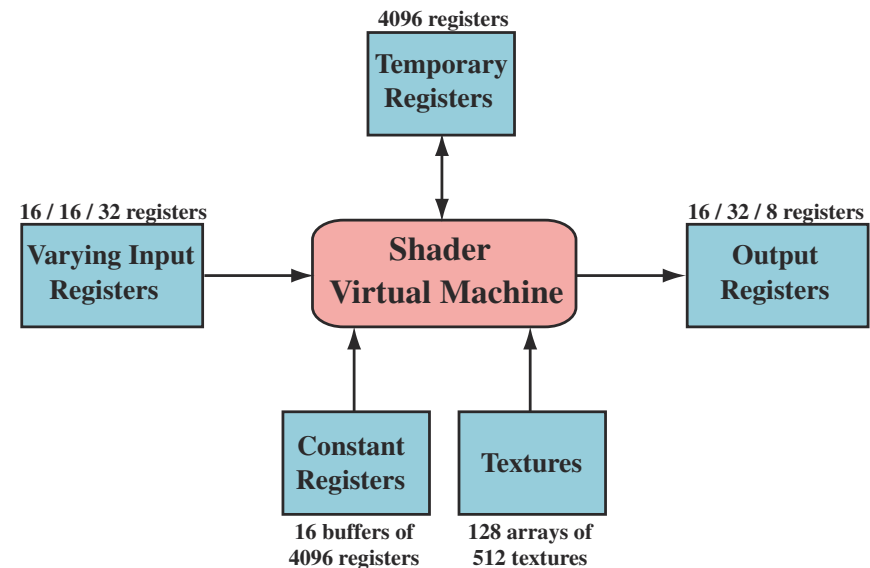
- Modern shader programs use a unified shader design
- Vertex, pixel, geometry and tessellation shaders share a common programming model
- The same instruction set is used - Unified shader architecture
- A pool of identical shader cores is allocated to different parts of the pipeline as seen fit
 - A mesh with finer triangles will need more vertex shaders than a mesh with larger ones.
 - A unified shader architecture can do load balancing while a fixed pool distribution cannot
- Shaders are programmed with C-like shading languages (HLSL, GLSL)
- HLSL can be compiled to intermediate language (IL or DXIL) and stored. Upon execution the graphics driver translates this IL to the specific instruction set of the hardware.

The Programmable Shader Stage

- Basic data types: 32-bit single-precision floating point scalars and vectors
- Modern GPUs also support 32-bit integers and 64-bit floats
- Floating point vectors typically contain positions and normals (xyzw), colors (rgba), matrix rows, texture coordinates (uvwq)
- The language model also supports aggregate data types (structures, arrays and matrices)
- A draw call to the API, to draw a group of similar primitives sets the machinery in motion

The Programmable Shader Stage

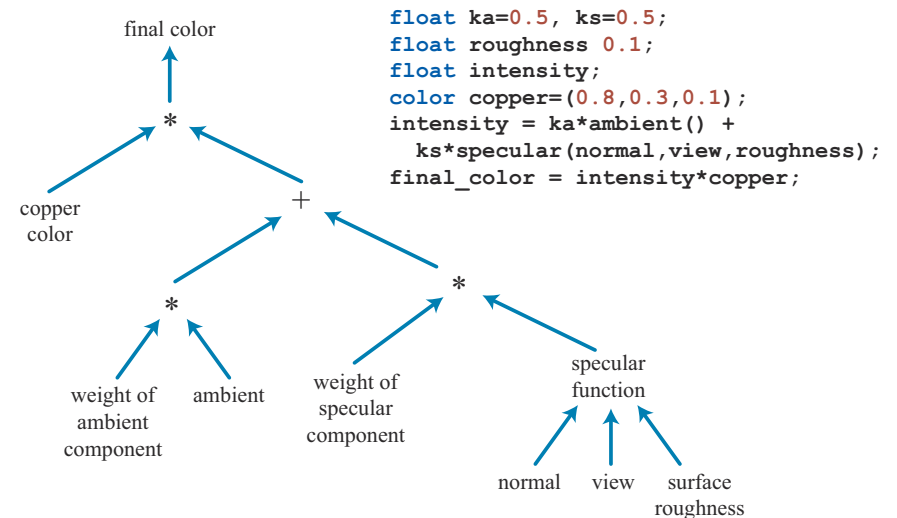
- Each programmable shader stage has two types of input:
 - Uniform inputs (remain constant throughout the draw call)
 - Varying inputs (coming from vertex data or from rasterization)
- Input and output register are fewer than constant and temporary registers.
- Efficient operations: *, /, intrinsic functions atan(), sqrt(), log(), vector normalization, reflection, dot and cross products, matrix transpose and determinants.
- 2 types of flow control: static (based on the values of uniform variables) and dynamic (based on varying inputs)
- Static flow control has no thread divergence, while dynamic flow control may have serious costs in performance.



Unified shader model 4.0 architecture and register layout

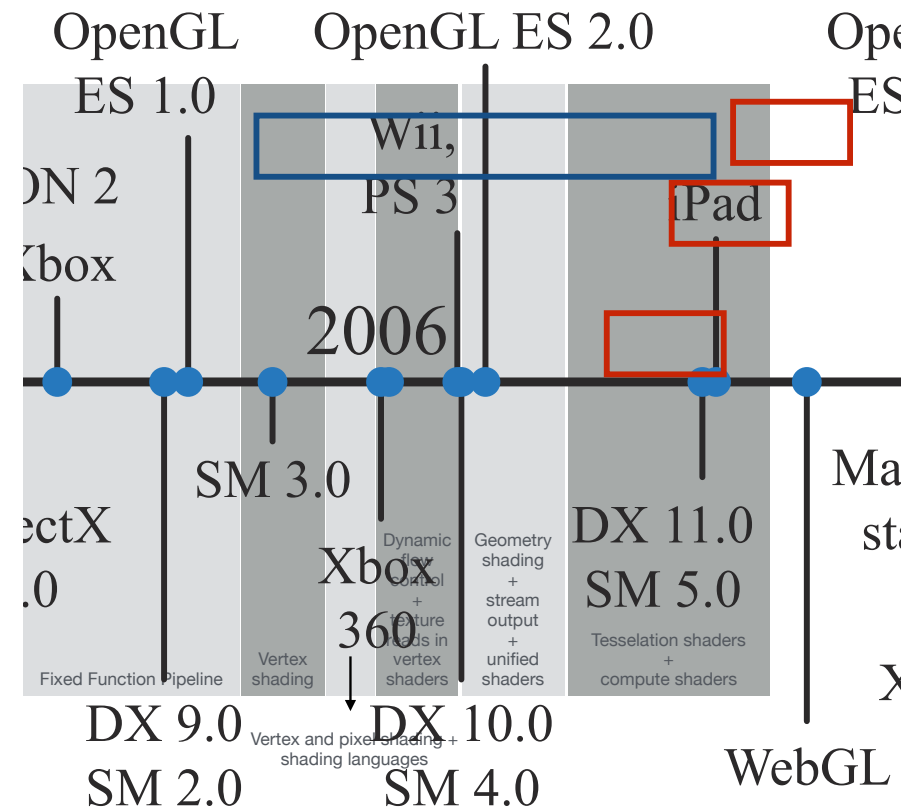
The Evolution of Programmable Shading and APIs

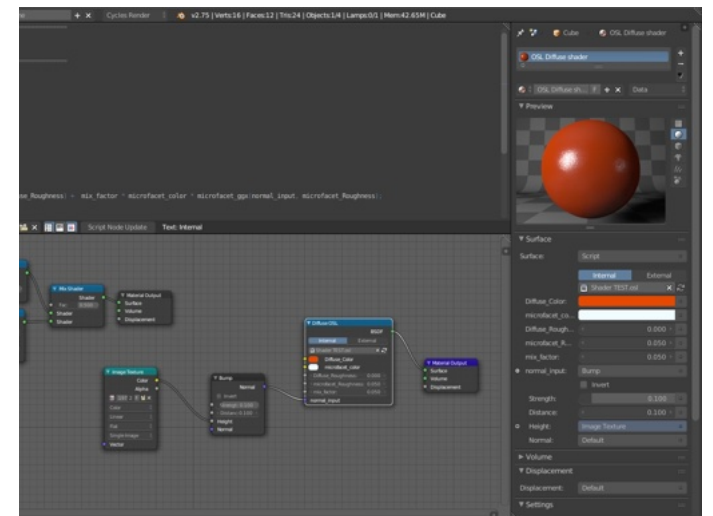
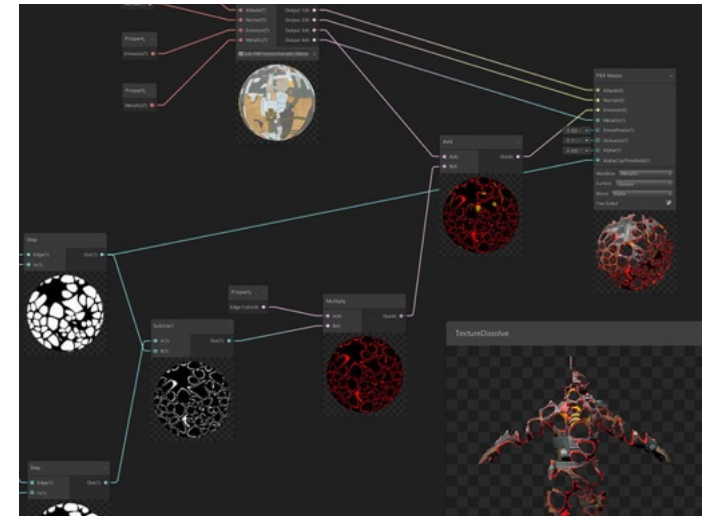
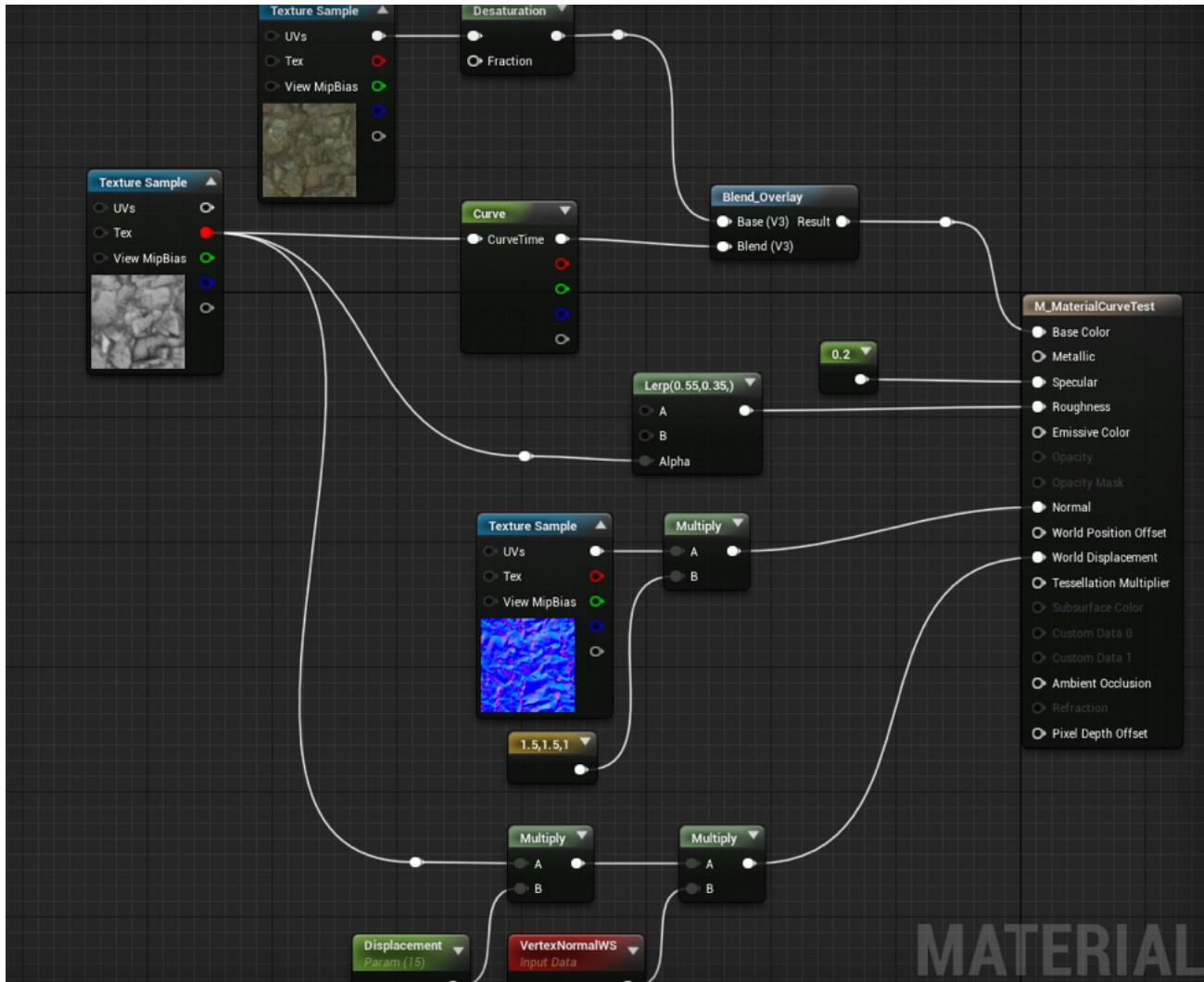
- Cook's shade trees [1984] were the first programable framework for shading
- RenderMan's shading language adopted the idea in the late 1980s. It is still used today for movie production
- OpenShadingLanguage is also inspired in this idea and is used in several products (Blender/Cycles, Sony Imageworks/Arnold, appleseed, V-Ray, ...)



The Evolution of Programmable Shading and APIs

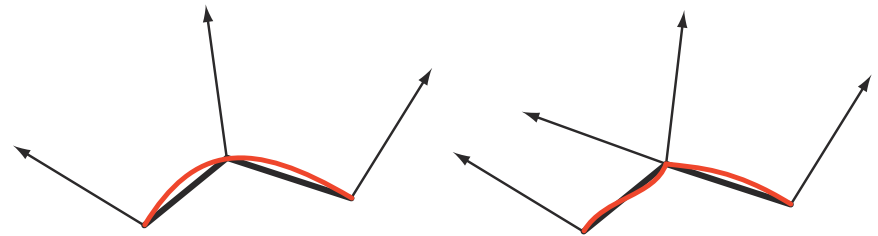
- Consumer level accelerated 3D graphics started with 3dfx VoodooGraphics Card [1996] (daughter card)
- First real programmable GPUs with DirectX 8.0 Vertex shading abilities written in assembly language. Still no branching and limited data types
- DirectX 9.0 [2002] programmable vertex and pixel shaders. Addition of texture reads, writing floating point values, static flow control and a shading language HLSL. OpenGL came up with GLSL
- Shader Model 3.0 added dynamic flow control and texture lookups in vertex shaders. More resources available to shaders (registers, shader length)
- DirectX 10.0 introduced SM 4.0 with geometry shaders, stream output and a unified shader model
- DirectX 11.0 + SM 5.0 introduced the tessellation shader and the compute shader (General Purpose GPU Computing)
- Mantle API [2013] intended to strip the graphics driver overhead and put the control in the developer, leading to a very low level API and better CPU multiprocessor support (Metal and Vulkan followed)
- DirectX 12 is an API redesign and also incorporates this low level API idea. No new functionality is exposed from previous 11.3 release
- Mobile device support is achieved through OpenGL ES lineage of releases + Metal (On Apple devices)

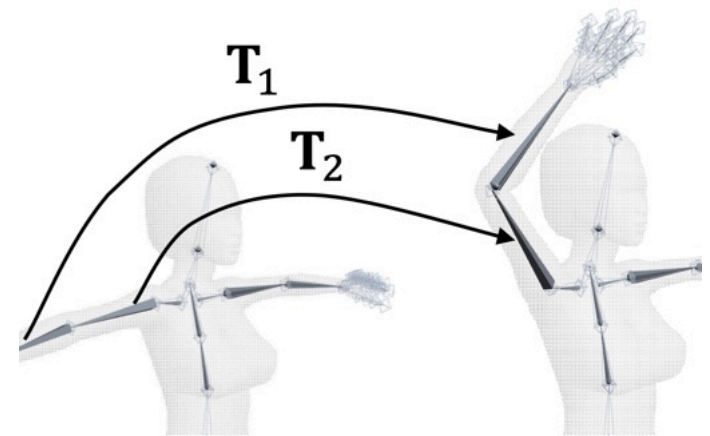
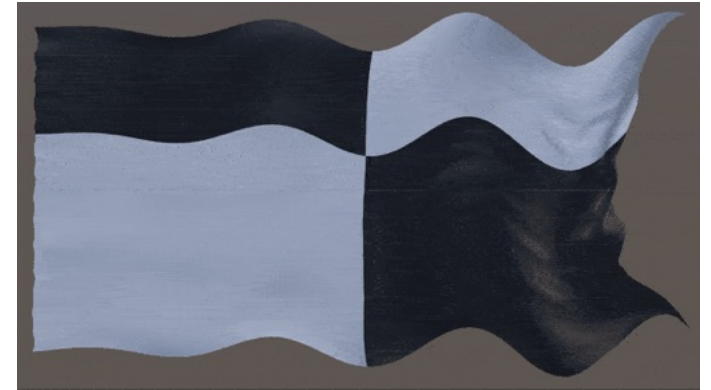




The Vertex Shader

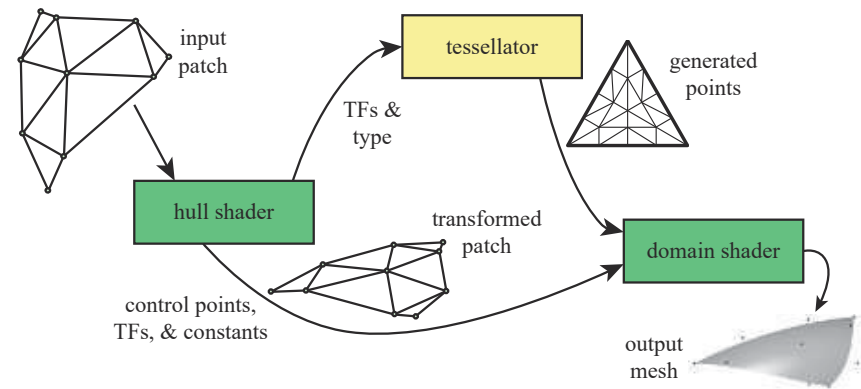
- It is the first programmable stage but some data manipulations take place before (feed the shaders with data from buffers and constant registers)
- Meshes are represented by a set of vertices, each with a specific position on the model surface. Vertex data may also include color, texture coordinates and surface normals (among others)
- The vertex shader processes the vertices of the mesh but without knowledge of the actual triangles in it
- From the set of inputs of each vertex, the shader **has to** output a final vertex position, along with optional data to be Interpolated across the primitive
- Typical behaviour is to go from model space to clip space (Model + View + Projection transformation)
- It cannot create nor destroy vertices and vertices are processed in an independent way
- The driver can decide how many GPU processors to allocate to process a stream of vertices



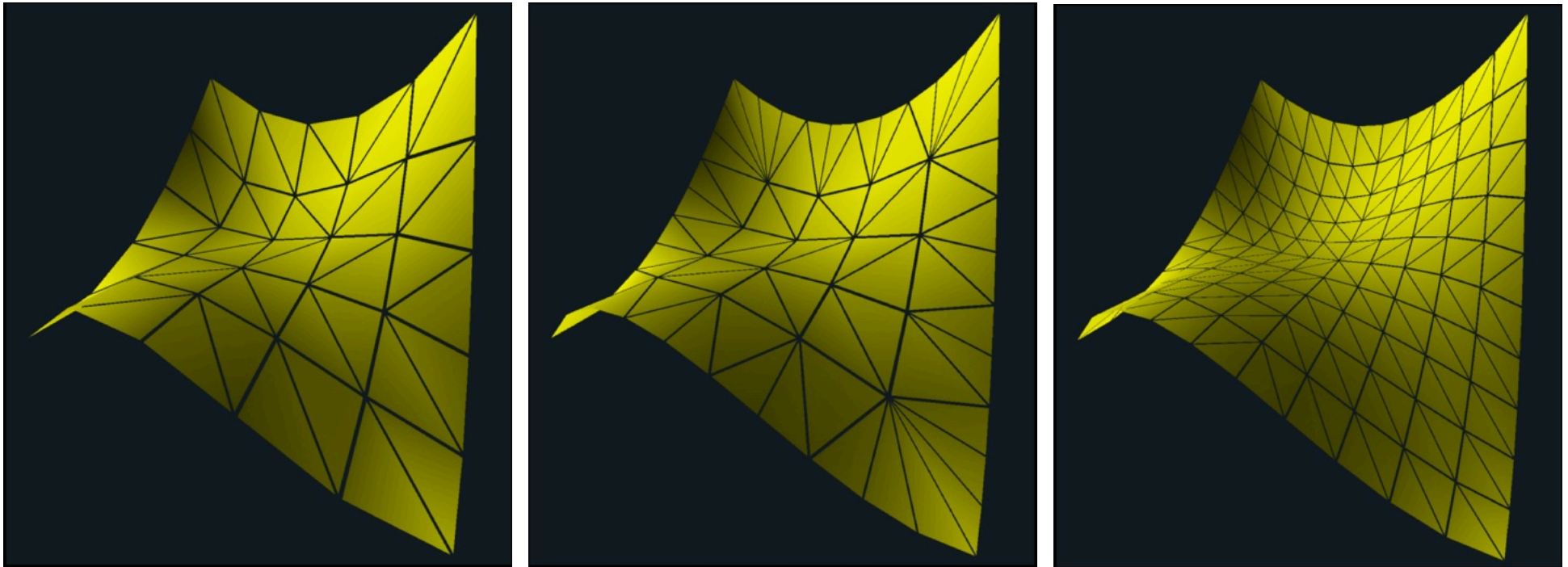


The Tessellation Stage

- Suitable to render curved surfaces
- Processes a patch (control mesh) and generates a more refined mesh
- Less stress on the bus between CPU and GPU
- Level of detail can vary depending on actual needs
- Comprises 3 stages:
 - Hull shader/Tessellation control shader
 - Tessellator/Primitive generator (fixed function)
 - Domain shader/Tessellation evaluation shader

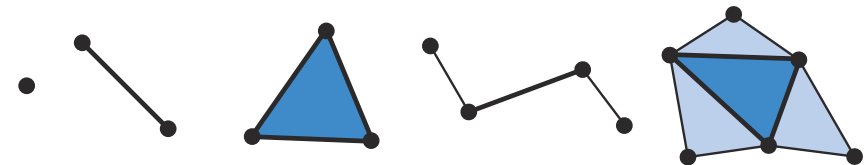


The Tessellation Stage



The Geometry Shader

- The geometry shader can turn primitives into other primitives
 - Triangle -> lines
 - Lines -> quads
 - Point -> quad
- Outputs 0+ vertices (treated as points, lines or strips of triangles)
- The least used from all the shaders (some mobile devices implement it in software as it is the more generic one and deviates from traditional shader model)



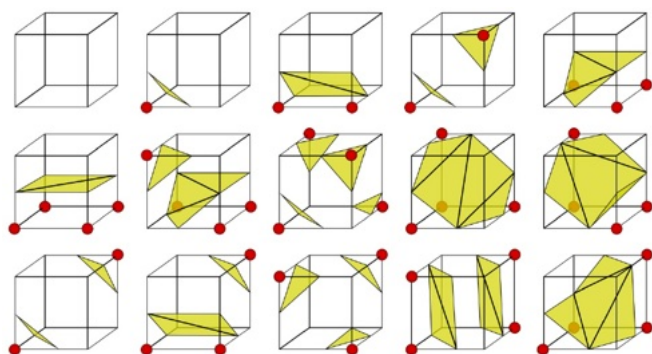
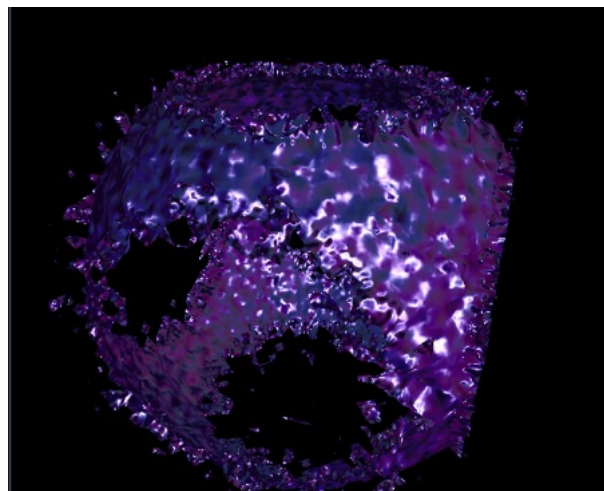
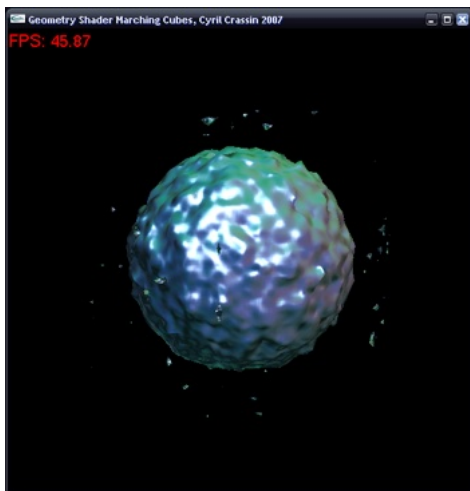
Inputs:

SM 4.0

point, line [+2 points], triangle [+3 points]

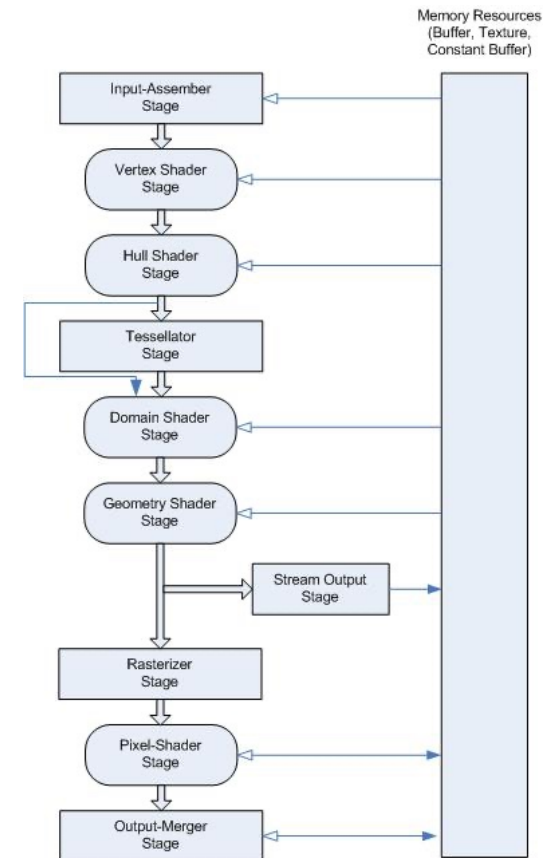
SM 5.0

Up to 32 control points



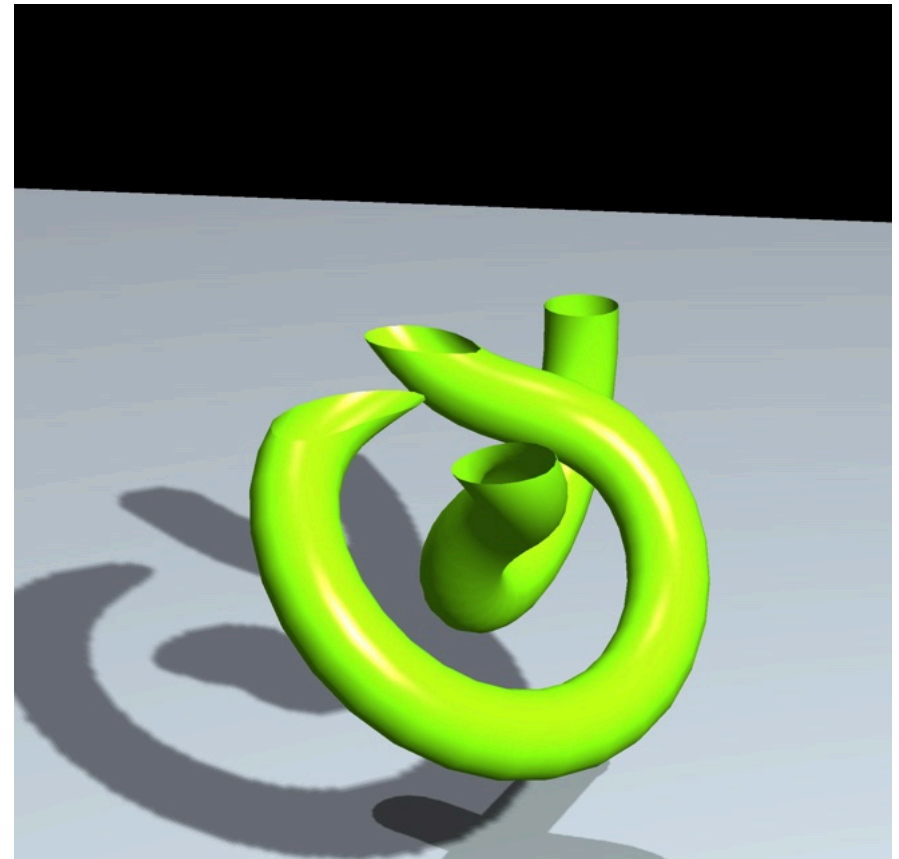
Stream Output

- SM 4.0 introduced the idea of using the first stages of the pipeline as a non-graphical stream processor [rasterization can even be turned off]
- Vertex shader [-> tessellation shader -> geometry shader] -> output stream
- The output stream (ordered list of vertices) could be sent back through the pipeline (iterative process)
- Stream output works on primitives (not on vertices) and outputs floating point values. A triangle will generate 3 vertices. If input data shares vertices, the share is lost
 - Normally it is used with point primitives
- In OpenGL this is called *transform feedback* since we transform vertices and return them to the start of the pipeline
- Primitives are sent to the output in the order they were received

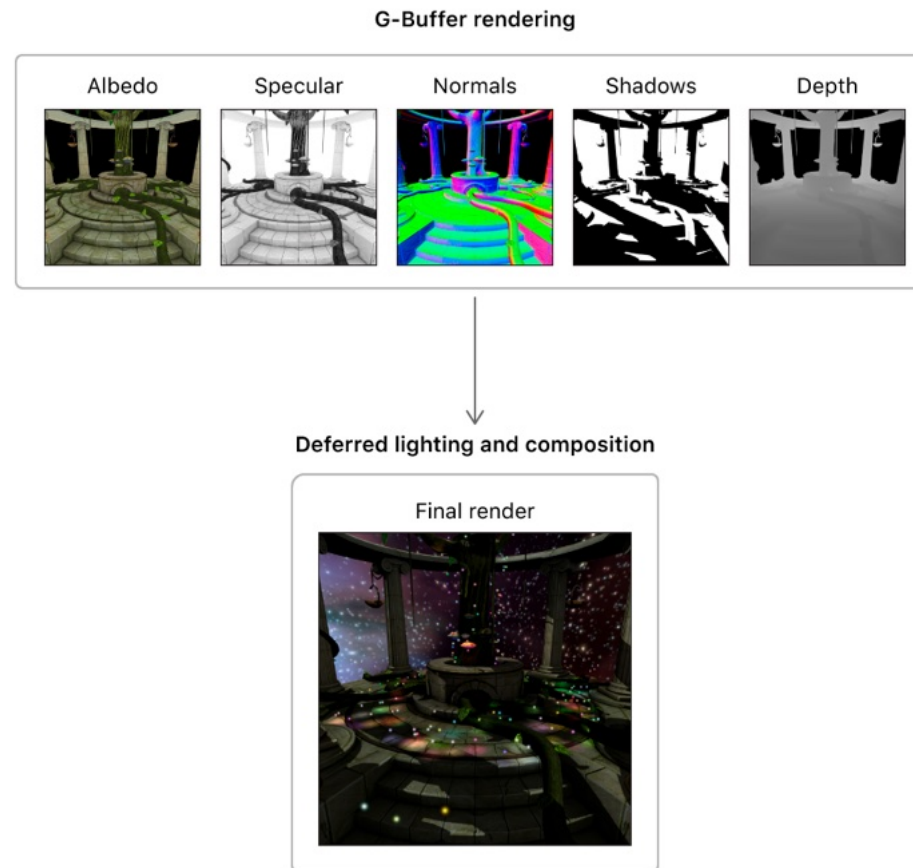


The Pixel Shader

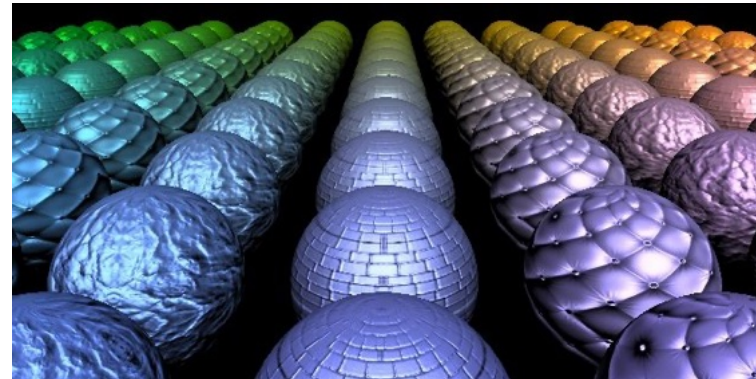
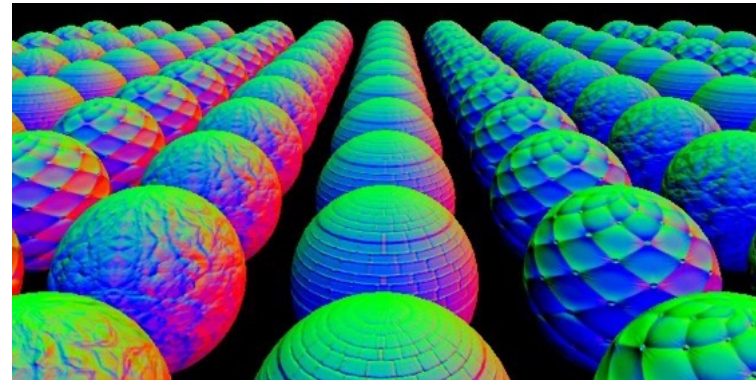
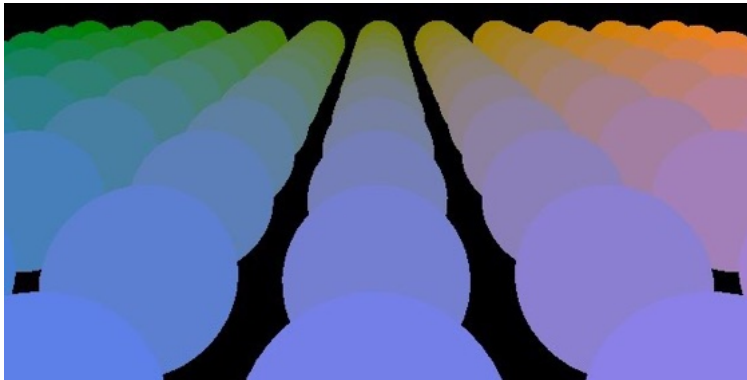
- The pixel shader is also known as fragment shader (a pixel that is only partially covered by the primitive)
- Its primary goal is to compute a final color (and opacity) for the pixel/fragment (or discard it)
- The pixel shader receives an interpolated z-value (it can be modified too)
- interpolated quantities can be perspective corrected (interpolated in World Space) or not (interpolated in screen space)
- The pixel shader inputs are the vertex shader outputs after interpolation
- Additional inputs are present (screen space position of the fragment, which side of the face it belongs to)
- It is possible to output to more than one render target
 - World space position, normal, object ids, etc...
 - Allow for alternative rendering pipelines such as deferred shading
- Cannot read adjacent pixel data (not directly at least)



Deferred Shading

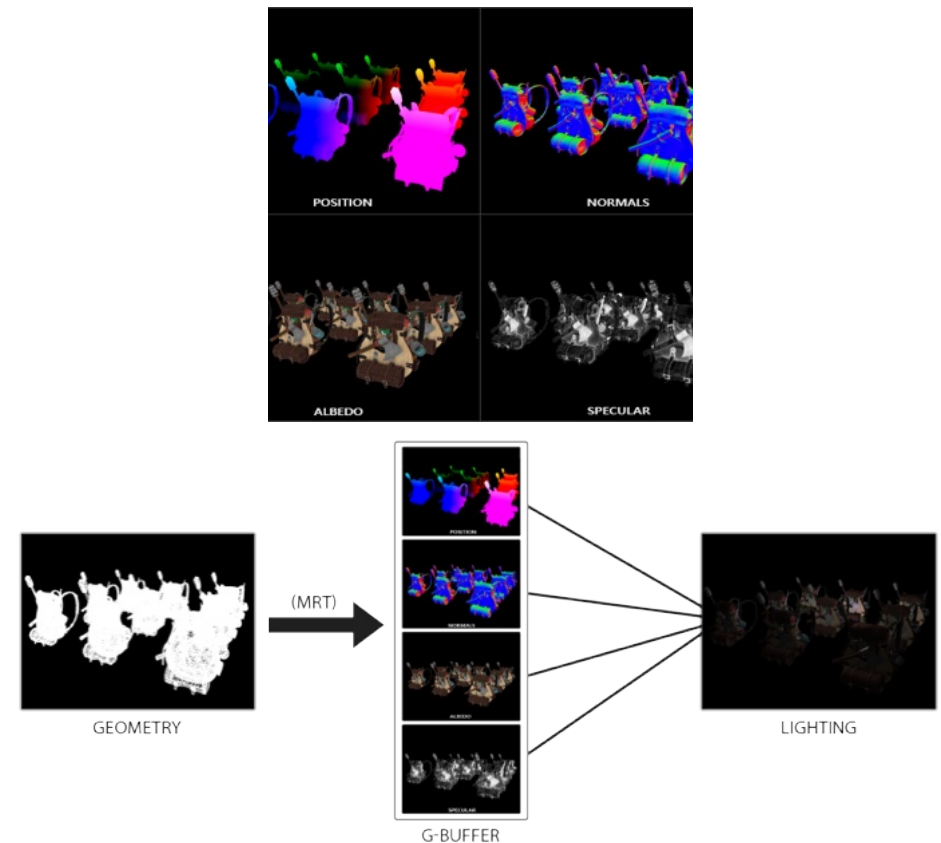


Deferred Shading



Deferred Shading

- The base idea is to defer or postpone most of the heavy rendering tasks to a later stage
 - 1st pass: Geometry pass - render all types of geometric information (positions, normals, colors (albedo), specular values, etc.) to separate textures (render targets). This is called a G-Buffer (Geometry Buffer)
 - 2nd pass: Lighting pass - render a quad filling the screen. For each fragment (visibility has already been determined), compute the lighting
- Pros: no wasted time computing expensive lighting on fragments that will not be visible
- Cons: No alpha blending
- It is possible to combine forward with deferred rendering
- This technique paves the way for rendering hundreds or thousands of light sources in a scene! How?

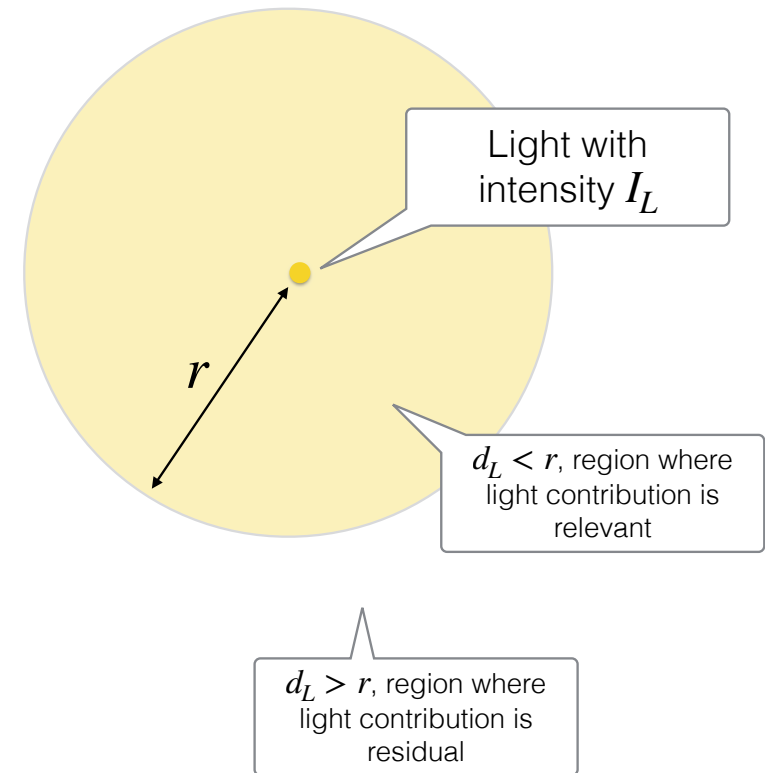


Deferred Shading and lots of lights

- Every light source has a limited radius. From light source attenuation term we can compute this radius by choosing a suitable lower threshold for I and solve for distance

$$I = \frac{I_L}{c_0 + c_1 d_L + c_2^2 d_L^2}$$

- Having the lighting stage shader iterate through each light source and check for its distance is doable but not very efficient
- It is better to draw spheres located at each light source, with the corresponding radius) in the lighting stage (one call per light source) and accumulate each light's contribution
- Two spheres touching each other should not occlude each other (depth write disabled)
- Only half of the pixels of a sphere should be considered and additive blending should be turned on
- We go from $\mathcal{O}(N_O \times N_L) \Rightarrow \mathcal{O}(N_O + N_L)$



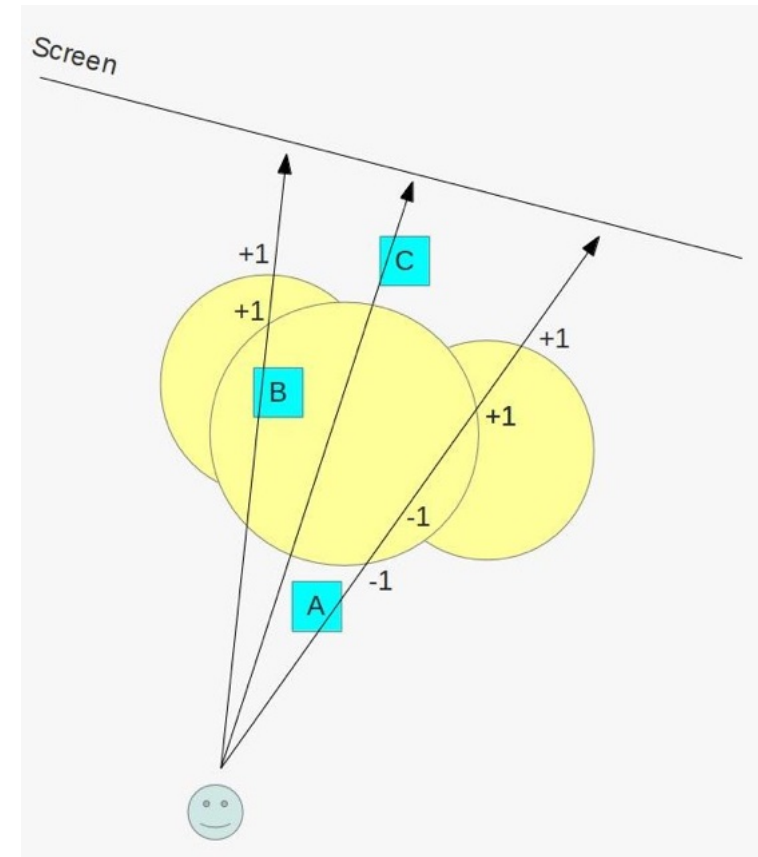
Stencil buffer

- The stencil buffer is a buffer with the same resolution as the framebuffer and the depth buffer, and typically with 1 byte per pixel.
- Stencil and depth tests control if the pixel is written to the frame buffer
- Application defines stencil test(s) and stencil operation(s)
- Possible stencil test functions:
 - Always pass
 - Always fail
 - Less/greater than
 - Less/greater or equal than
 - Equal
 - Not equal
- Possible stencil operation:
 - Keep the stencil unchanged
 - Replace with value, Replace with 0
 - Increment/decrement (with/without wrapping around)
 - Invert the bits
- Stencil operations are configured independently for these situations:
 - Stencil test failure
 - Depth test failure
 - Depth test success
- Stencil tests and operations can be independently set for each face side (front/back)

Deferred Shading and lots of lights (details)

- Render objects as usual into the G-Buffer, including the depth buffer data
- Disable writing into the depth buffer
- Disable back face culling
- Set the stencil test to always succeed (we just need the stencil operation here)
- Set stencil operation for back facing polys to increment when depth test fails; otherwise no change
- Set stencil operation for front facing polys to decrement when depth test fails; otherwise no change
- Render sphere for light with null pixel shader
- Finally render the sphere for the light again and set the stencil test to pass if different from 0. This stage adds the light contribution to the previous lighting stage with additive blending.

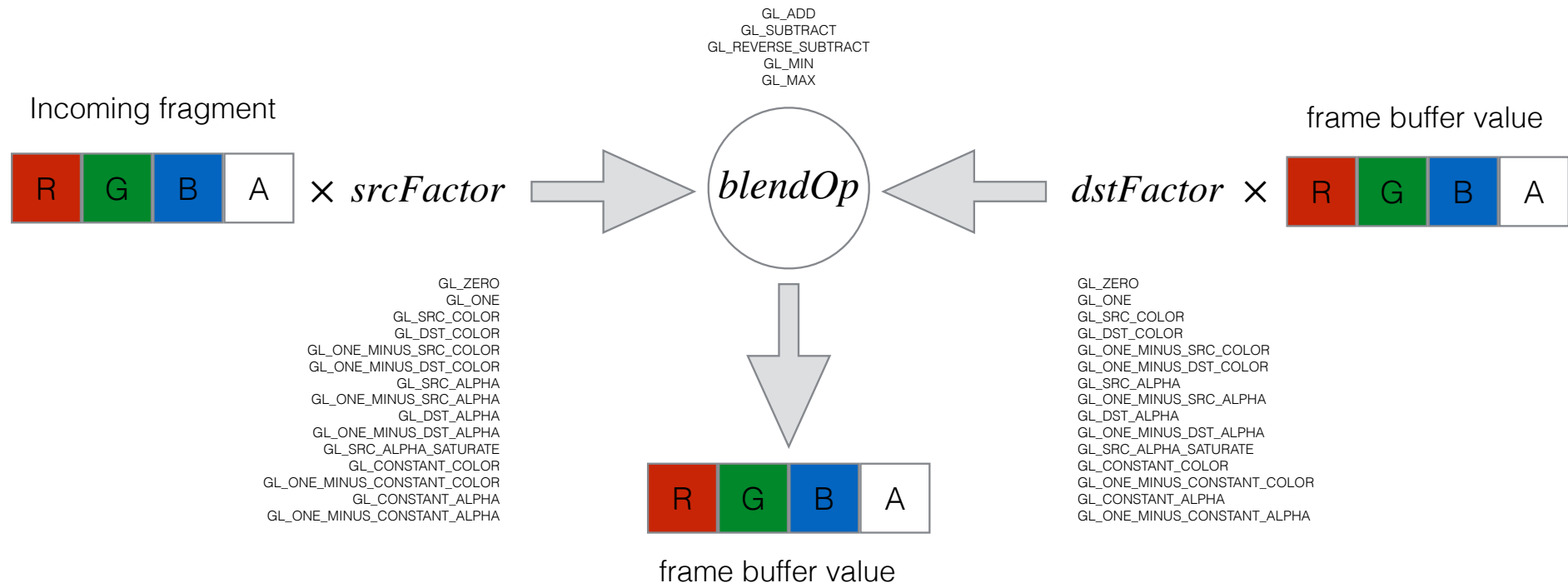
Note: This will also work for several light sources



The Merging Stage

- Combines depths and colors of the individual fragments
- Stencil buffer and z-buffer operations occur in this stage
- If a fragment is visible it gets blended with the existing color or replaces it
- Some GPUs allow some merging operations to be performed before the pixel shader gets executed (avoid spending time computing colors for non visible fragments)

Alpha Blending



Over operator (back to front order): $A_{src}RGB_{src} + (1 - A_{src})RGB_{dst}$

Under operator (front to back order): $C_{dst} = A_{dst}(A_{src}C_{src}) + C_{dst}$

Note: It is possible to choose different (srcFactor, dstFactor) combinations for RGB and ALPHA channels

$$A_{dst} = 0 + (1 - A_{src})A_{dst}$$

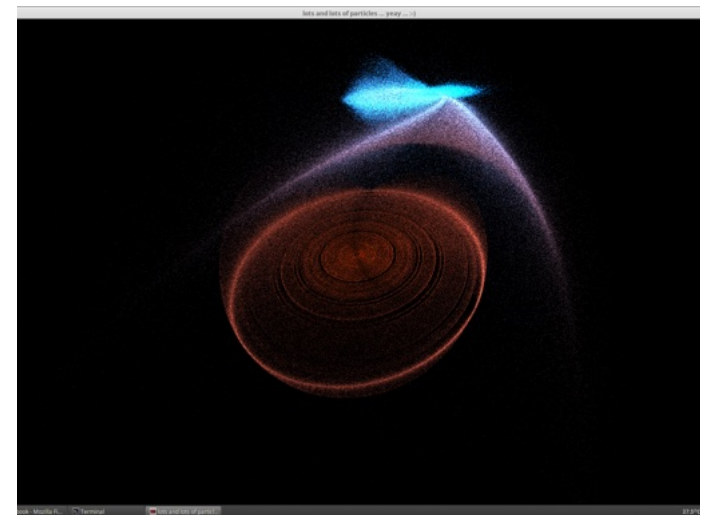
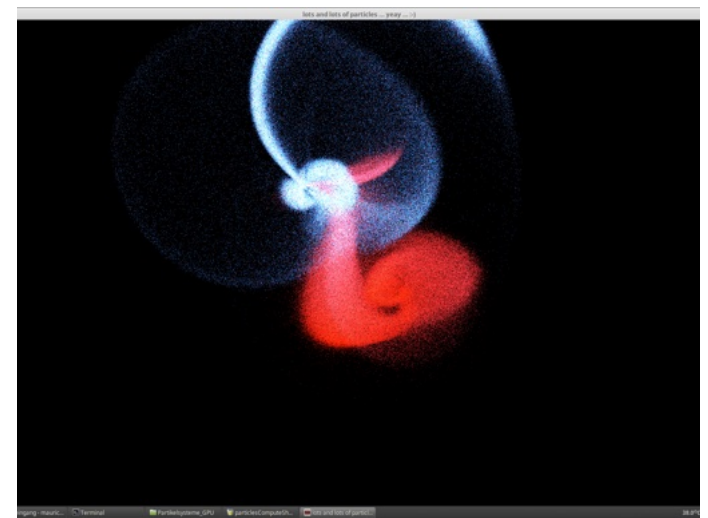
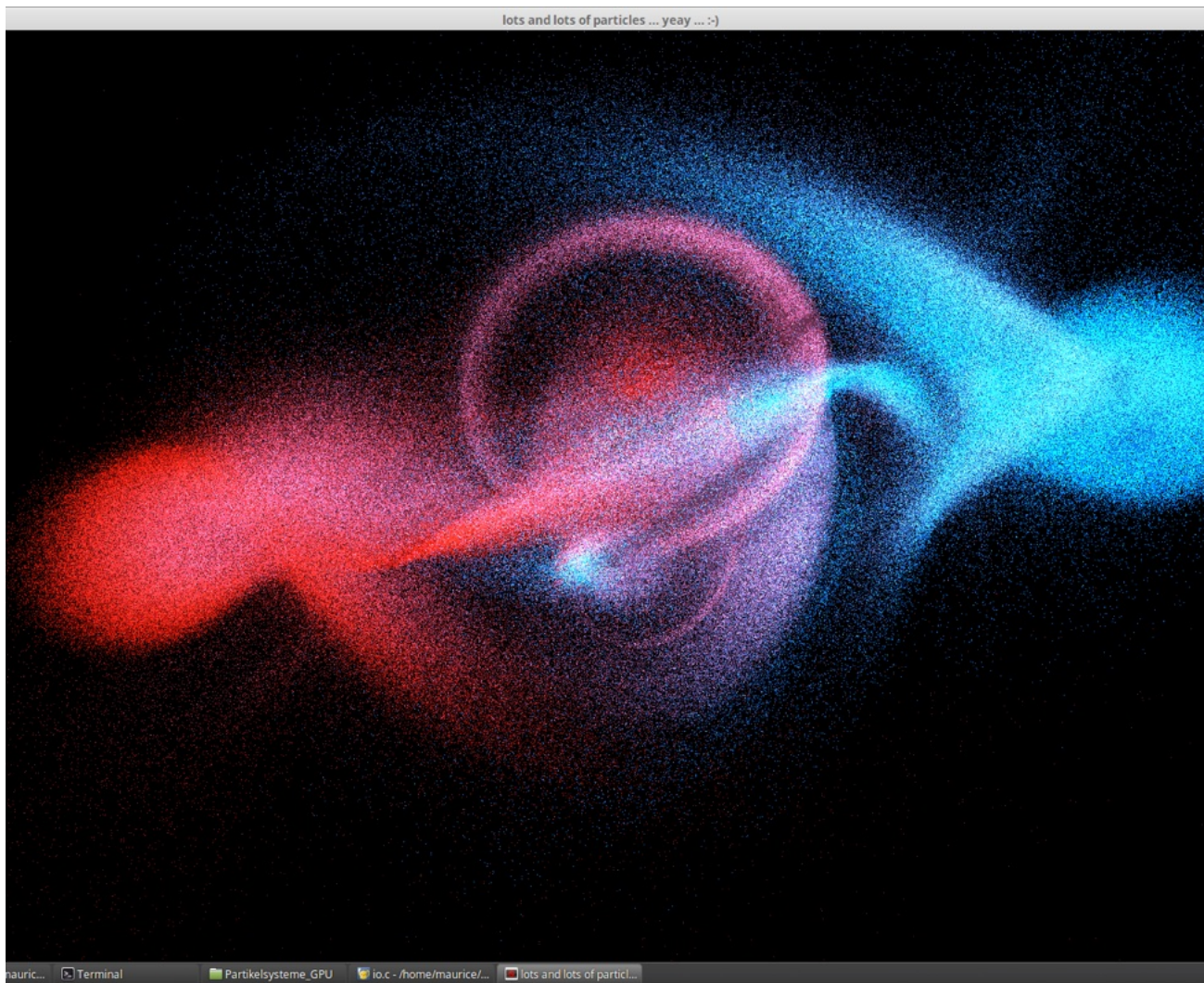
$$C_{dst} = A_{dst}C_{bg} + C_{dst}$$

The Compute Shader

- The GPU can also be used to perform general computations
- Application areas where GPUs are commonly used:
 - Financial markets (stock order predictions)
 - Neural networks and deep learning
 - Physics simulation, particle systems
 - Virtual currency
- CUDA (Compute Unified Device Architecture) and OpenCL (Open Computing Language) are the most widely used platforms (languages, compilers and libraries) to explore the power of the GPUs for general purpose computing.

The Compute Shader

- The Compute Shader was introduced in DirectX 11
- It is a shader that is not locked to a specific part of the graphics pipeline, is invoked by the graphics API and it shares the resources (pool of shader processors) used in the pipeline.
- It receives input data, can access buffers (such as textures) for input and output. Threads and warps (thread bundles running on a same processing unit) are visible in compute shaders
- Each invocation of a compute shader receives a thread index.
- Thread groups offer x- y- and z- coordinates in a grid and share resources among threads, such as fast shared memory (typically 32KB in DirectX).



Compute Shader (Example)

```
#version 430 core
```

```
// Process particles in blocks of 128
layout (local_size_x = 128, local_size_y = 1, local_size_z = 1) in;

layout (std430, binding = 0) buffer PositionBuffer {
    vec3 positions[];
};
layout (std430, binding = 1) buffer VelocityBuffer {
    vec4 velocities[];
};
layout (binding = 2) buffer AttractorBuffer {
    vec4 attractors[];
};
layout (std430, binding = 3) buffer LifeBuffer {
    float lifes[];
};
// Delta time
uniform float dt;

highp float rand(vec2 co)
{
    highp float a = 12.9898;
    highp float b = 78.233;
    highp float c = 43758.5453;
    highp float dt= dot(co.xy ,vec2(a,b));
    highp float sn= mod(dt,3.14);
    return fract(sin(sn) * c);
}

float vecLen (vec3 v)
{
    return sqrt(v.x*v.x + v.y*v.y + v.z*v.z);
}

vec3 normalize (vec3 v)
{
    return v / vecLen(v);
}

vec3 calcForceFor (vec3 forcePoint, vec3 pos)
{
    // Force:
    float gauss = 10000.0;
    float e = 2.71828183;
    float k_weak = 1.0;
    vec3 dir = forcePoint - pos.xyz;
    float g = pow (e, -pow(vecLen(dir), 2) / gauss);
    vec3 f = normalize(dir) * k_weak * (1+ mod(rand(dir.xy), 10) - mod(rand(dir.yz), 10)) / 10.0 * g;
    return f;
}
```

```
void main(void)
{
    uint index = gl_GlobalInvocationID.x;

    int i;
    float newDT = dt * 100.0;

    vec3 forcePoint = vec3(0);

    for (i = 0; i < 32; i++) {
        forcePoint += attractors[i].xyz;
    }

    // Read the current position and velocity from the buffers
    vec4 vel = velocities[index];
    vec3 pos = positions[index];
    float newW = lifes[index];

    float k_v = 1.5;

    vec3 f = calcForceFor(forcePoint, pos) + rand(pos.xz)/100.0;

    // Velocity:
    vec3 v = normalize(vel.xyz + (f * newDT)) * k_v;

    // Eine leichte Anziehung richtung Schwerpunkt...
    v += (forcePoint-pos) * 0.00005;

    // Pos:
    vec3 s = pos + v * newDT;

    newW -= 0.0001f * newDT;

    // If the particle expires, reset it
    if (newW <= 0) {
        s = -s + rand(s.xy)*20.0 -rand(s.yz)*20.0;
        //v.xyz *= 0.01f;
        newW = 0.99f;
    }

    lifes[index] = newW;
    // Store the new position and velocity back into the buffers
    positions[index] = s;
    velocities[index] = vec4(v,vel.w);
}
```

Example taken from: https://github.com/MauriceGit/Partikel_accelleration_on_GPU

Further readings and resources

- Cap. 3 Real Time Rendering - T Akenine-Möller et. Al (adopted book)
- Unity Grass Shader Tutorial - <https://roystan.net/articles/grass-shader.html>
- Deferred Shading Tutorial - <http://ogldev.atspace.co.uk/www/tutorial35/tutorial35.html> + <https://catlikecoding.com/unity/tutorials/rendering/>
- Particle System Compute Shader - https://github.com/MauriceGit/Partikel_acceleration_on_GPU

Advanced Shading

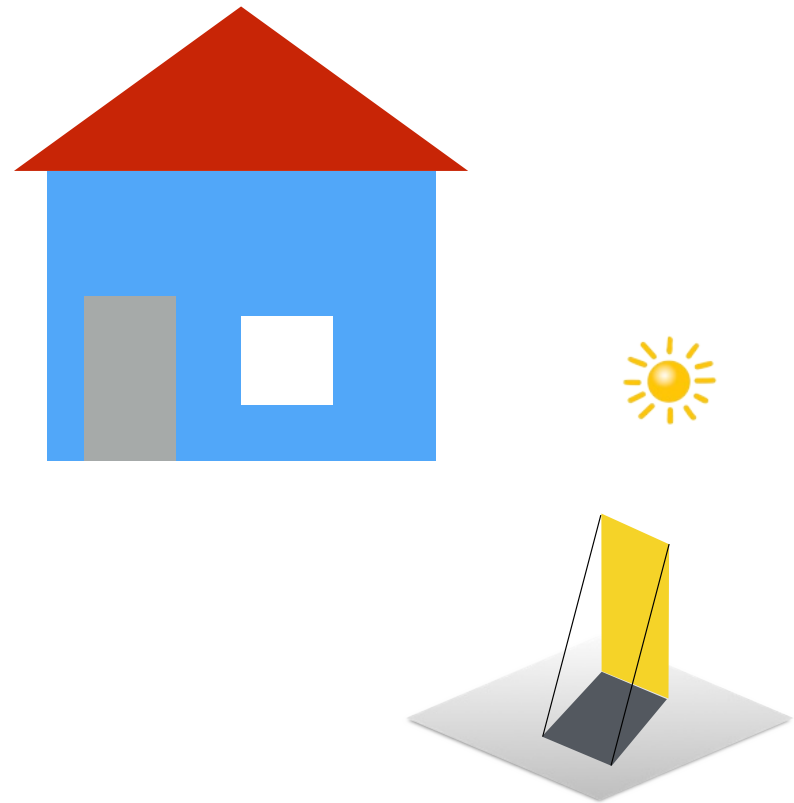
Surface Detail

- Most real surfaces don't look smooth or uniform
- Surface detail is missing!
- Several techniques can be used to add detail to surfaces...



Surface Detail Polygons

- create features on surfaces by adding polygons (doors, windows, lettering, shadows) to a base polygon
- surface-detail polygons need to be given priority in visible surface determination
- surface-detail polygons are rendered and made visible instead of the base polygon, thus adding detail



Texture Mapping

The finer the detail, the less viable it is to add surface-detail polygons...

It is better to map an image (digitised or synthesised) onto the surfaces

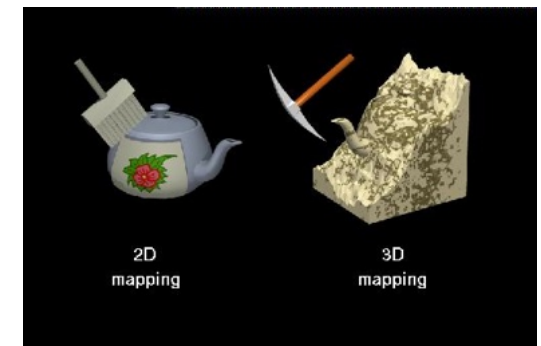
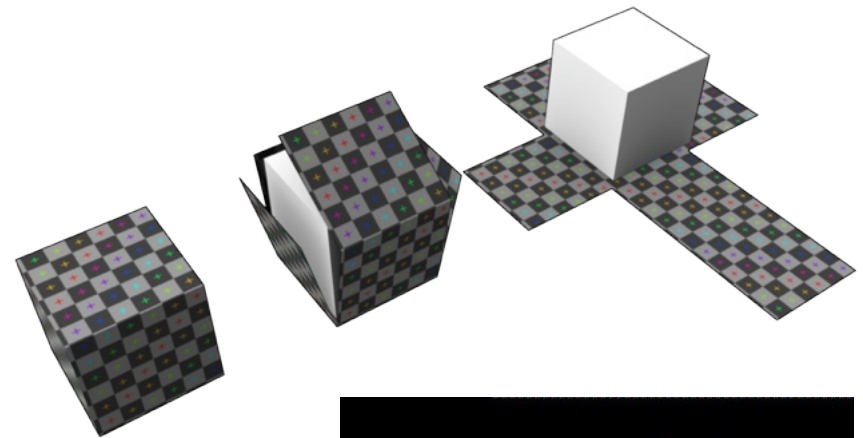
How many surface-detail polygons would be needed for this image?



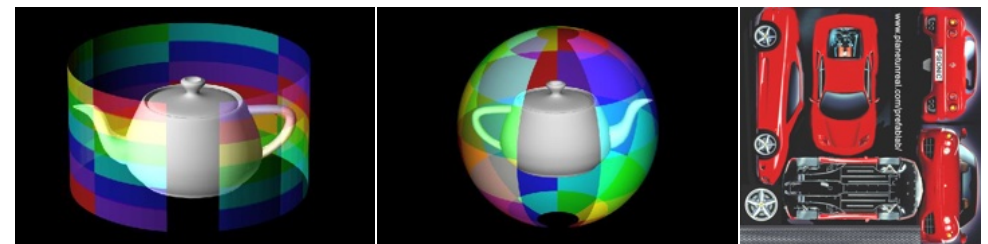
*from Unity3D Forum

Texture Mapping

- Mapping can be done in 2D (painting) or 3D (sculpting).
- Textures can be digitised or synthesised
- Textures can be used for almost anything related to shading, and not only to replace object's colour!
- What type of mapping to use?
- What will we do with the looked up value?

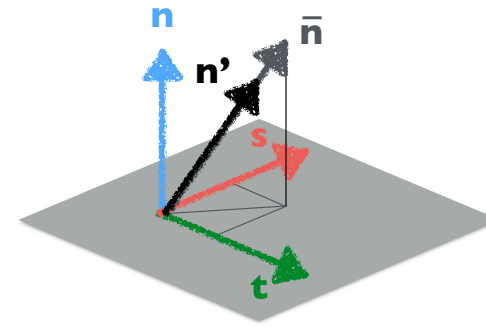


*from SIGGRAPH.org education materials



Bump Mapping

- Bump mapping perturbs surface normals according to a bump map (texture)
- Surface geometry is not perturbed, only lighting
- Bump maps are implementation dependent. For instance, we can use just the R and G values to tilt the normal vector
- We will need a pair of mutually perpendicular vectors $\mathbf{s}$ and $\mathbf{t}$, tangent to the surface.



$$\bar{\mathbf{n}} = \mathbf{n} + R\mathbf{s} + G\mathbf{t}$$

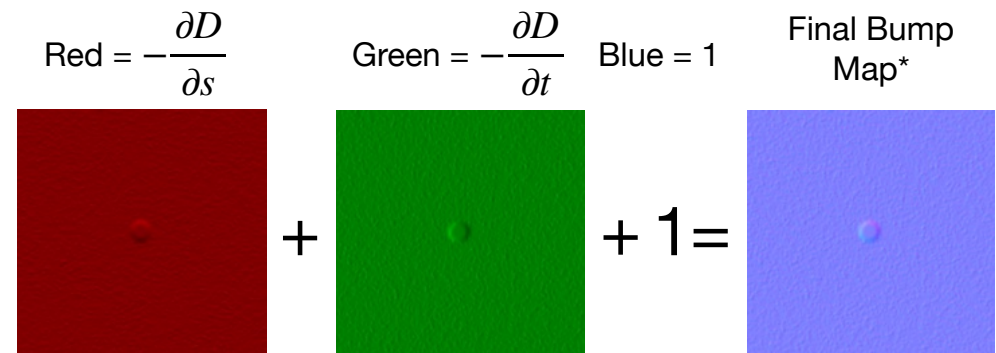
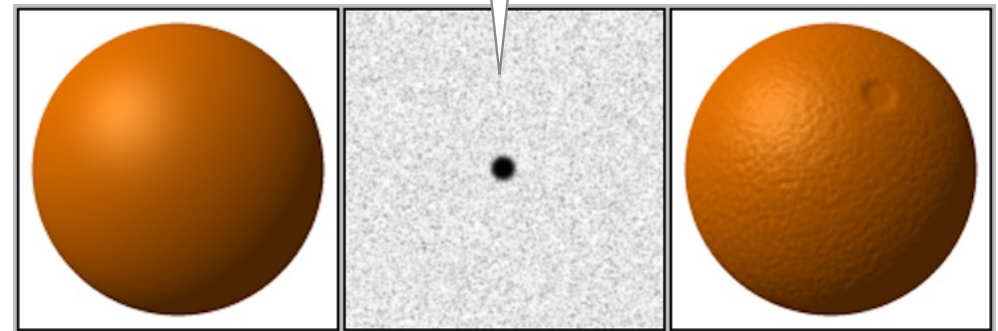
Perturbed normalised normal: $\mathbf{n}' = \frac{\bar{\mathbf{n}}}{\|\bar{\mathbf{n}}\|}$

R and G are the values obtained from the Red and Green channels of the bump texture to tilt the final normal vector along $\mathbf{s}$ and $\mathbf{t}$. Bump map values need to be mapped to $[-1,1]$ range.

Bump Mapping

- Displacement map creates a displaced surface: $\mathbf{p}'(s, t) = \mathbf{p}(s, t) + D(s, t)\hat{\mathbf{n}}$
- Tangent vectors of the displaced surface:
 - $\partial \mathbf{p}' / \partial s = \partial \mathbf{p} / \partial s + \partial D / \partial s \hat{\mathbf{n}}$
 - $\partial \mathbf{p}' / \partial t = \partial \mathbf{p} / \partial t + \partial D / \partial t \hat{\mathbf{n}}$
- It can be shown that the components of the normal vector in the tangent directions (s and t) are directly proportional to the gradients of the Displacement map (stored in the red and green channels).

Displacement map D - contains displacements along the original normal. Black (0) is mapped to a maximum negative displacement along the normal



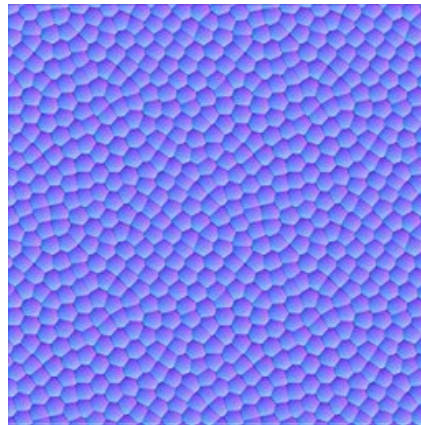
* usually implemented like this

Bump Mapping



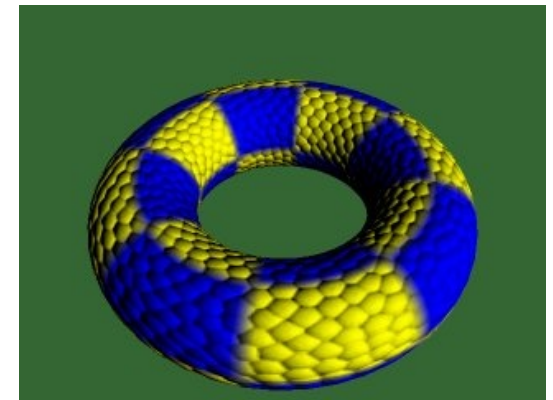
original textured model

+



normal map

=



final bump mapped model

A more direct approach is to store n'_s , n'_t and n'_z (in tangent space) directly in RGB channels of the bump map texture. This normal will be the one used. A (0,0,1) in tangent space value will produce the original normal.

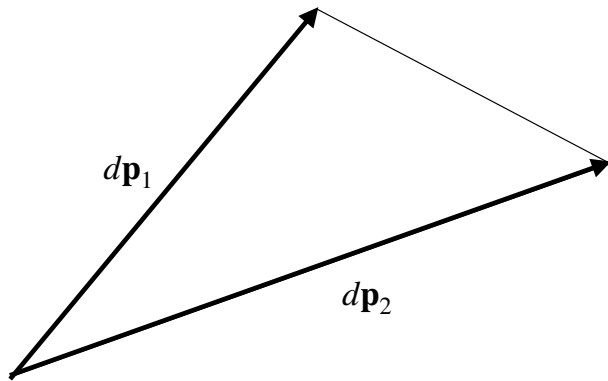


Tangent space

Tangent space is easily computed using the original normal vector and the directions of the texture coordinates. The normal vector coordinates (texture) are provided with the mesh, and uv coordinates also.

Bump mapping (bi-tangent vectors)

Edges e_1 and e_2 of a triangle correspond to the changes in positions $d\mathbf{p}_1$ and $d\mathbf{p}_2$, and to the changes in texture coordinates (du_1, dv_1) , and (du_2, dv_2) , respectively.



Assemble the system:

$$\begin{cases} d\mathbf{p}_1 = du_1 \mathbf{T} + dv_1 \mathbf{B} \\ d\mathbf{p}_2 = du_2 \mathbf{T} + dv_2 \mathbf{B} \end{cases}$$

Solve for the unknowns $\mathbf{T}$ and $\mathbf{B}$!

$\mathbf{T} = (T_x, T_y, T_z)$ and/or $\mathbf{B} = (B_x, B_y, B_z)$ can be computed in object coordinates and stored as vertex attributes

Bump mapping (bi-tangent vectors)

$$\begin{bmatrix} T_x & B_x & N_x \\ T_y & B_y & N_y \\ T_z & B_z & N_z \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} T_x \\ T_y \\ T_z \end{bmatrix}$$

$$\begin{bmatrix} T_x & B_x & N_x \\ T_y & B_y & N_y \\ T_z & B_z & N_z \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} B_x \\ B_y \\ B_z \end{bmatrix}$$

$$\begin{bmatrix} T_x & B_x & N_x \\ T_y & B_y & N_y \\ T_z & B_z & N_z \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} N_x \\ N_y \\ N_z \end{bmatrix}$$

Converts from tangent space to model space

$$\begin{bmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \\ N_x & N_y & N_z \end{bmatrix} \begin{bmatrix} T_x \\ T_y \\ T_z \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \\ N_x & N_y & N_z \end{bmatrix} \begin{bmatrix} B_x \\ B_y \\ B_z \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \\ N_x & N_y & N_z \end{bmatrix} \begin{bmatrix} N_x \\ N_y \\ N_z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Converts from model space to tangent space*

* lighting can be performed in tangent space too, if we also convert light and view direction vectors

Transparency

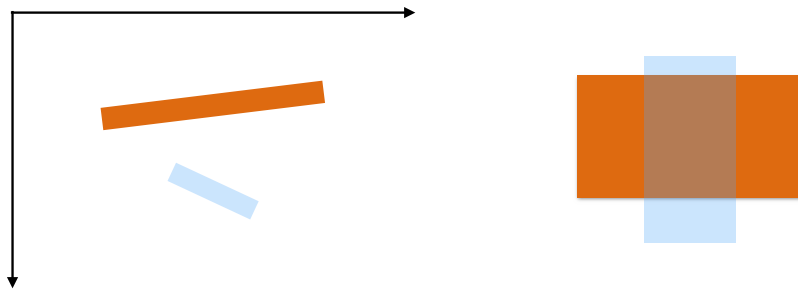


"Glasses" by Gilles Tran (2006), using povray

JOGOS E AMBIENTES IMERSIVOS - GAMES AND IMMERSIVE ENVIRONMENTS

Fernando Birra
Rui Nóbrega

Transparency (No Refraction)



Transparent **polygon 1** in front of opaque **polygon 2**

- Without refraction, light passes through transparent objects without being “bent”
- Interpolated transparency (alpha blending):

$$I_{\lambda} = \underbrace{(1 - k_{t_1})}_{\text{Opacity of polygon 1}} I_{1_{\lambda}} + k_{t_1} I_{2_{\lambda}}$$

Opacity of polygon 1

Transmission coefficient =
transparency of polygon 1

- Filtered transparency:

$$I_{\lambda} = I_{1_{\lambda}} + k_{t_1} O_{t_{\lambda}} I_{2_{\lambda}}$$

Allows for surface shading
(reflection/emission) of the object in
front.

Polygon's transparency colour
(Coloured filters will have different
values for each wavelength)

- In a pure filtering setup (no shading on the transparent objects) order of drawing is irrelevant...
- Otherwise... back-to-front order is required!

Transparency (No Refraction)

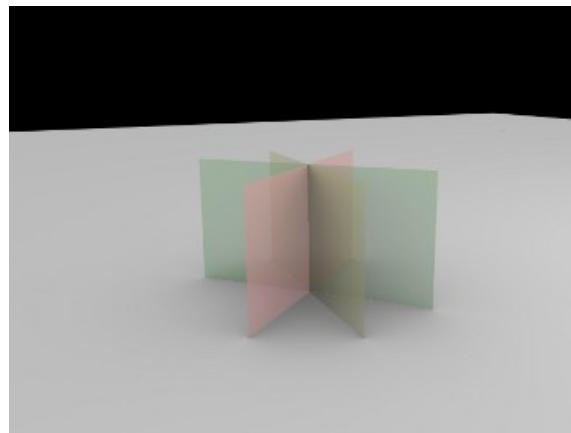
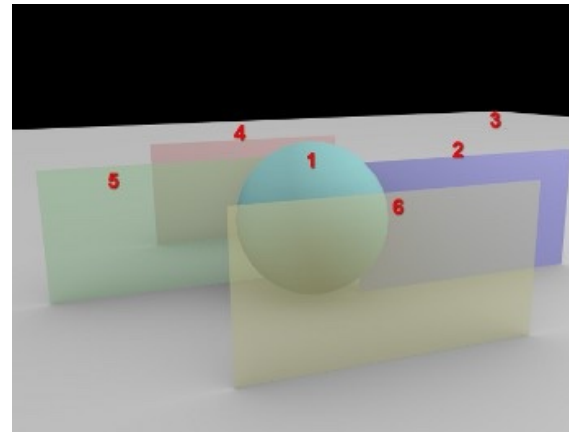
1. Draw all opaque objects using traditional z-buffer
2. Create 4 additional buffers (for transparent objects) with the following, for each pixel:
 - transparency value
 - flag (initially off)
 - colour
 - depth (initially closest possible value)
3. Render each transparent object and, for each pixel:
 - if its z value is closer than opaque z-buffer (visible) and more distant than current transparent z-buffer (farthest at the moment), then set flag and save colour, transparency and transparent z-value (depth).
4. When all objects have been processed, the transparent object buffers have the information about the more distant transparent objects at each pixel (where the flag value is set).
5. Update opaque buffers by blending colour and z-buffer from transparent buffers (see previous slide)
6. Repeat the process [steps 3-6] to render closer objects while the flag buffer has been modified at least once.

Transparency

objects 1, 2, 3 and 4 are opaque and rendered first

object 5 is transparent and needs to be rendered before object 6, also transparent

Object depth sorting could solve the problem in this example



In this case all polygons are transparent (apart from the plane) and there is no object sorting valid for all pixels

Either split each transparent polygon by its intersection lines with other polygons and perform object sorted rendering or...

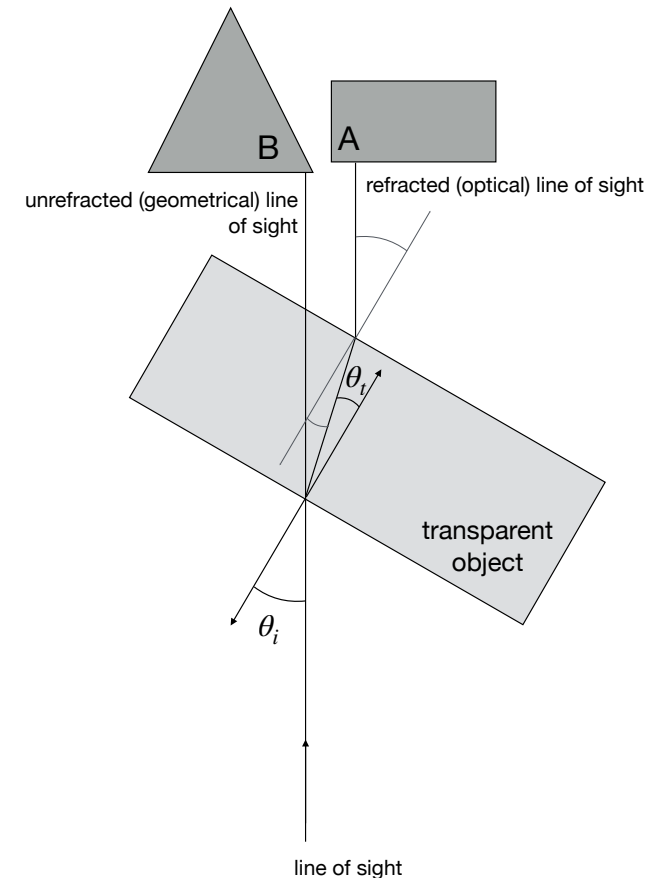
... apply the algorithm presented before.

Transparency (Refraction)

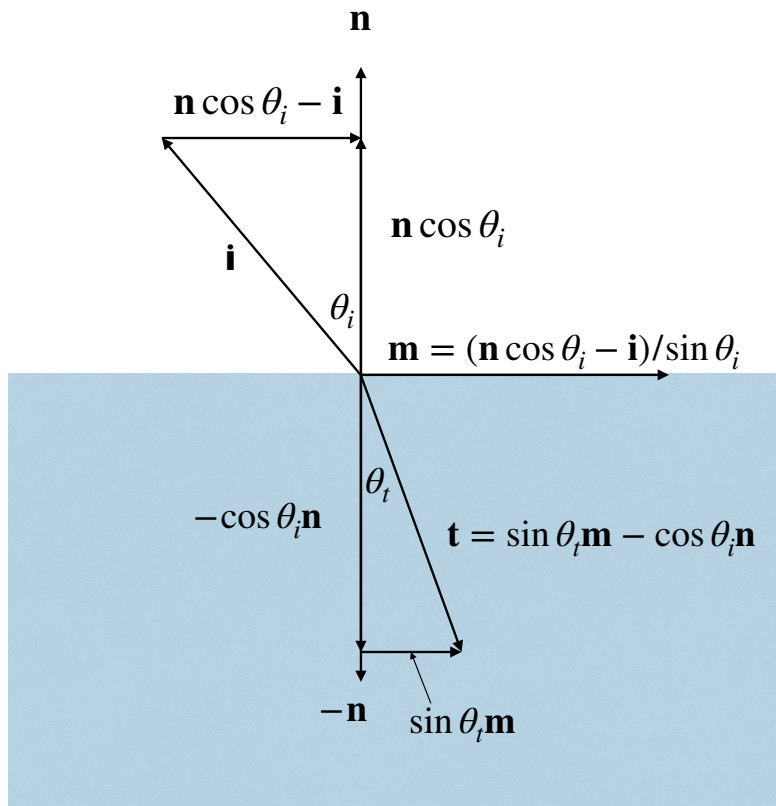
- Without refraction, object B would be visible along the geometrical line of sight
- With refraction, object A will be visible along that line of sight
- Light rays “bend” according to **Snell's law**:

$$\frac{\sin \theta_i}{\sin \theta_t} = \frac{\eta_{t_\lambda}}{\eta_{i_\lambda}}$$

η_λ - ratio between speed of light at certain wavelength λ in a vacuum and its speed inside the medium. All materials have values greater than 1.



Refraction Vector



Unit vector along transmitted direction:

$$\mathbf{t} = \sin \theta_t \mathbf{m} - \cos \theta_t \mathbf{n}$$

replacing $\mathbf{m}$:

$$\mathbf{t} = \frac{\sin \theta_t}{\sin \theta_i} (\mathbf{n} \cos \theta_i - \mathbf{i}) - \cos \theta_t \mathbf{n}$$

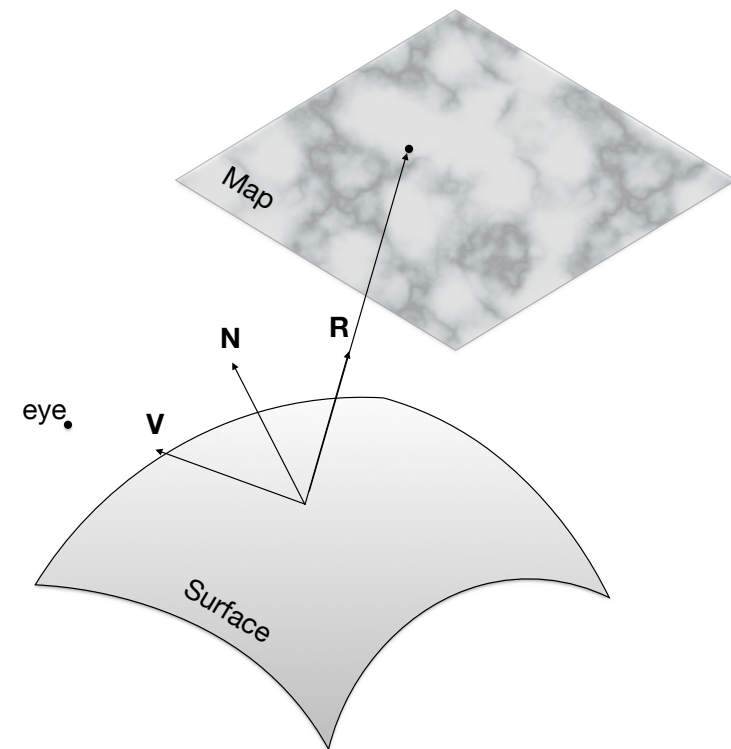
or:

$$\mathbf{t} = \frac{\eta_i}{\eta_t} (\mathbf{n} \cos \theta_i - \mathbf{i}) - \cos \theta_t \mathbf{n}$$

Inter-object Reflections

Reflection Maps

- Inter-object reflections (both specular and diffuse) occur when surfaces reflect other surfaces
- Reflection mapping (or environment mapping) model specular inter-object reflection
- mapping each vertex of a polygon will give the corresponding reflection polygon in the map
- The way **R** is used to index the map depends on the particular method.



Inter-object Reflections

Reflection Maps: spherical mapping

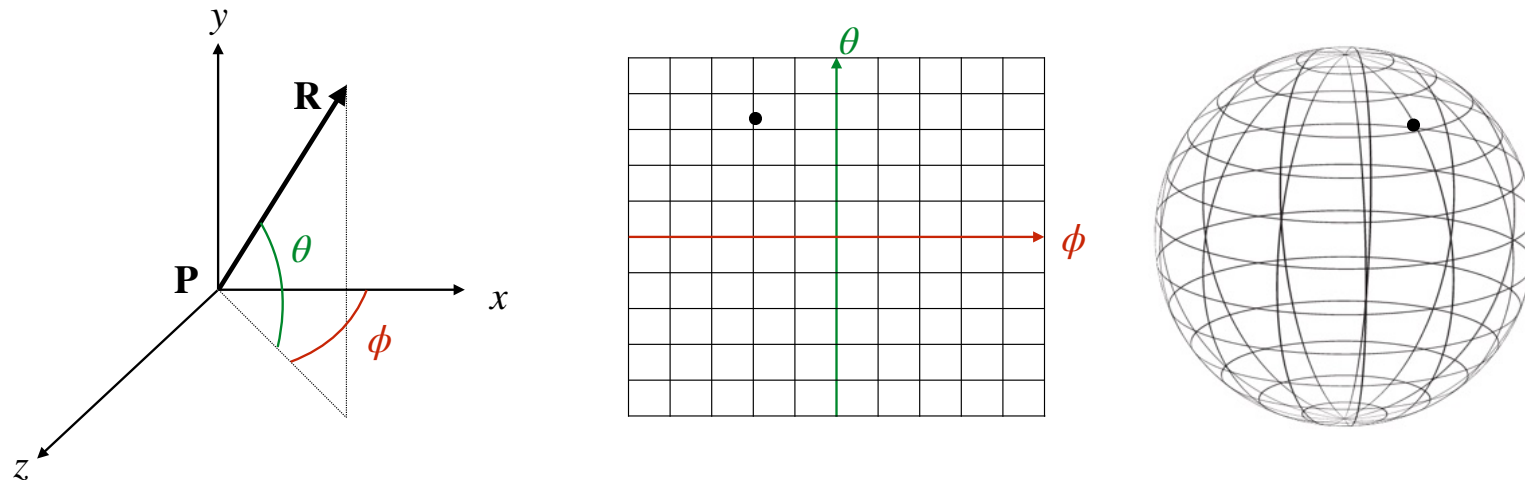
Spherical environment mapping starts by choosing a centre of projection to map the environment to be reflected onto a sphere surrounding the objects to be rendered

- The environment can be treated as a 2D texture
- On each point of the object to be rendered, the view vector $\mathbf{V}$ is reflected about $\mathbf{N}$ to give $\mathbf{R}$.
- $\mathbf{R}$ is used to index the environment map by using polar coordinates
- The environment mapped sphere is assumed to be infinitely big
 - Problems at the caps of the sphere
 - Distortion
 - Not valid for concave objects (no self reflections are visible)



Inter-object Reflections

Reflection Maps: spherical mapping



- The reflection vector (x, y, z) coordinates are mapped to polar coordinates (θ, ϕ) .
- Latitude (θ) and longitude (ϕ) are used to index the sphere map

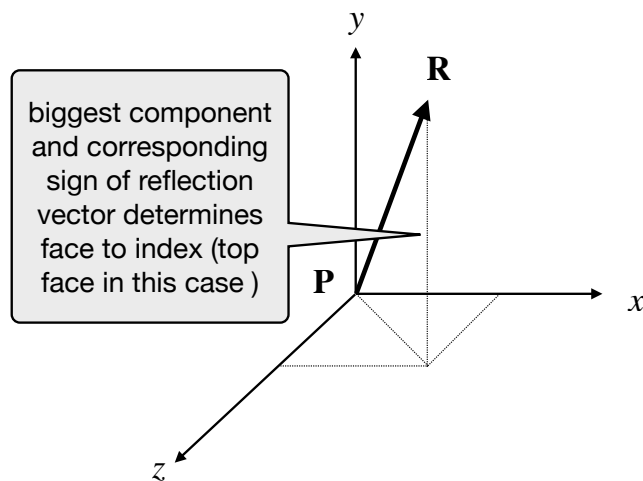
Inter-object Reflections

Reflection Maps: cube mapping

- Cube mapping is preferable to spherical mapping because:
 - ✓ causes less distortion
 - ✓ texture resolution is less critical (each face has the same resolution as the whole map of spherical mapping)
- A map is generated for each face of the cube by projecting the scene along the direction of that face
- Using the normalised reflection vector $\mathbf{R}$, the component with the highest magnitude tells which pair of opposite faces to look for. The sign chooses between the pair.

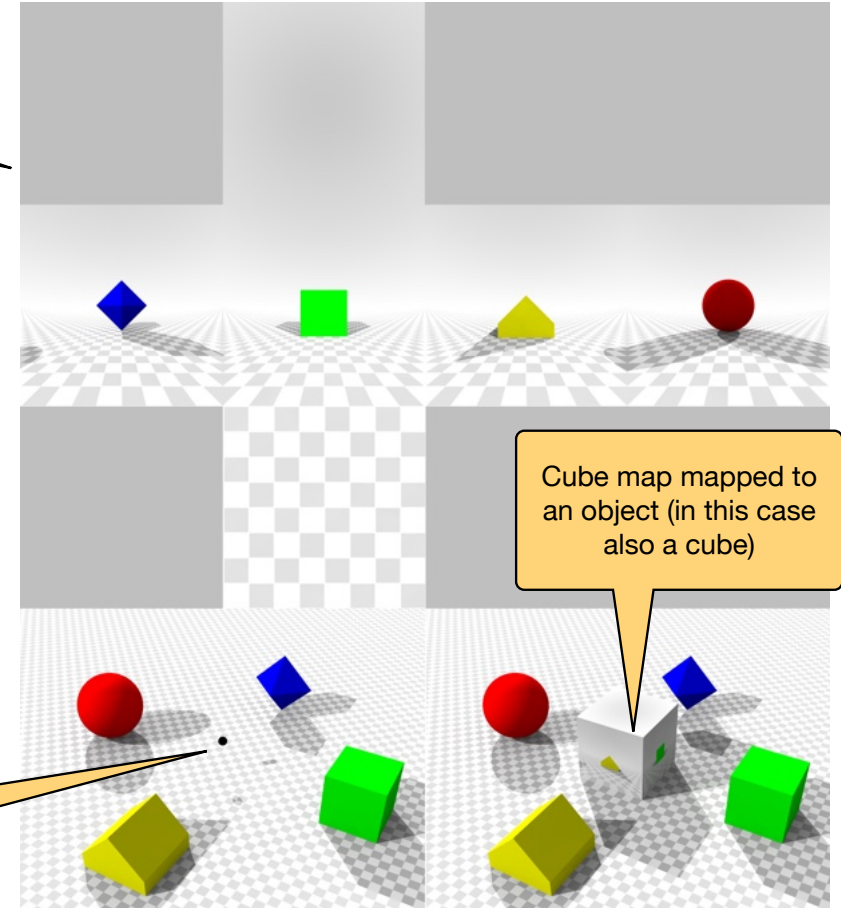
Inter-object Reflections

Cube mapping



Cube map

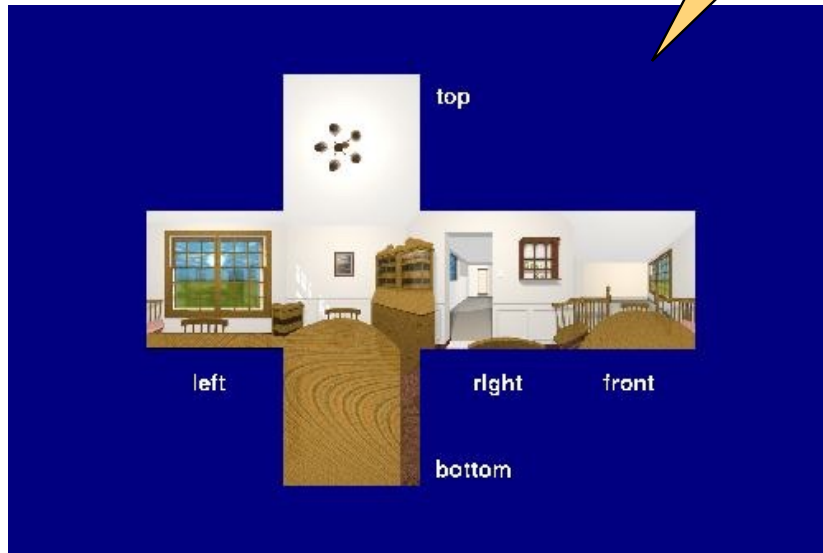
viewpoint to cube map generation



Inter-object Reflections

Cube Mapping

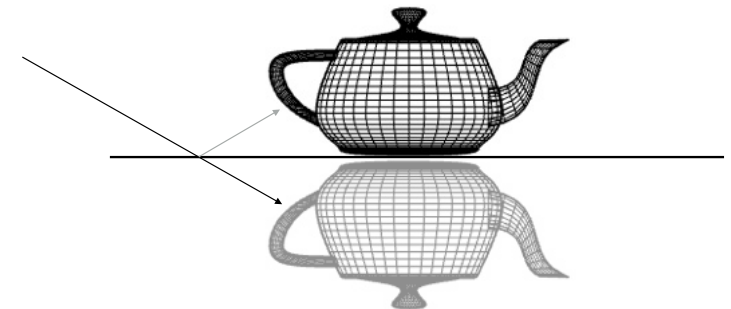
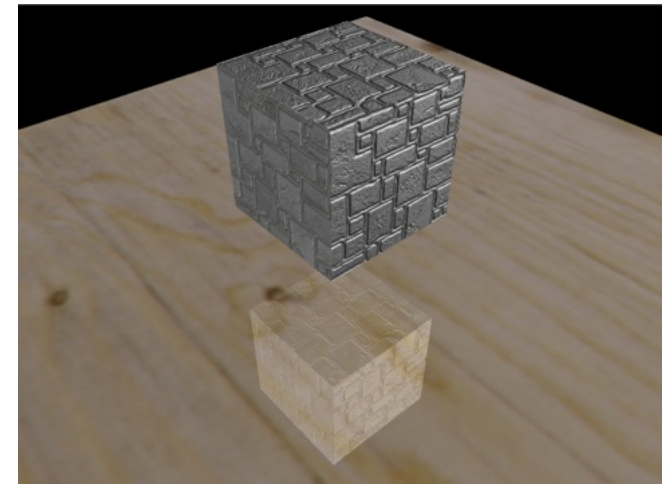
What if the objects in the scene are moving or if new objects are being added?



Inter-object Reflections

Planar Reflections

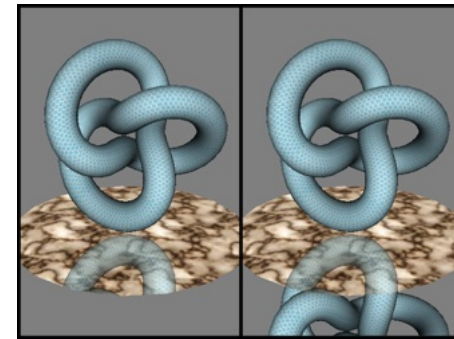
- Reflections on large planar surfaces are better handled by rendering an inverted (mirrored) version of the object
- Steps:
 1. Draw inverted model with z-buffer active
 2. Draw mirror plane with z buffer active and blend it with the frame buffer contents
 3. Draw normal model with z-buffer and no blending



Inter-object Reflections

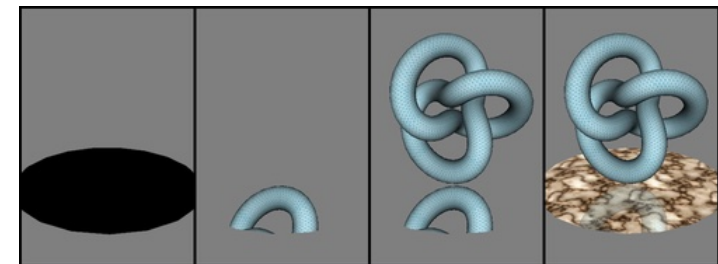
Planar Reflections

- What if the mirror plane is not big enough?
- **Revised Steps:**
 1. Render the mirror plane into a stencil buffer only
 2. Render the mirrored version with stencil enabled
 3. Clear depth buffer and render actual model
 4. Render the mirror plane using front-to-back blending



Right

Wrong



1

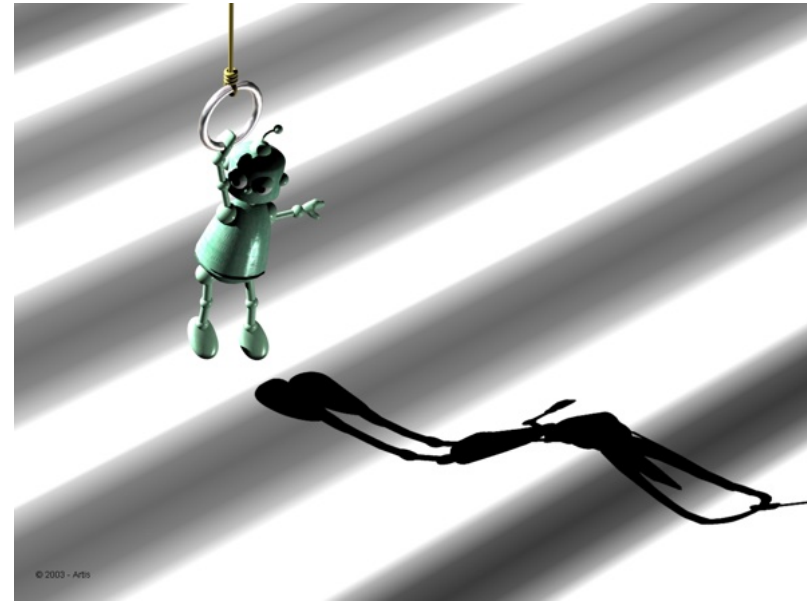
2

3

4

Shadows

Where is the character?



Shadows improve our perception of relative positions!

Shadows

- Shadows are relatively easy to include in shading, for point light sources
- Small change to Phong's illumination model:

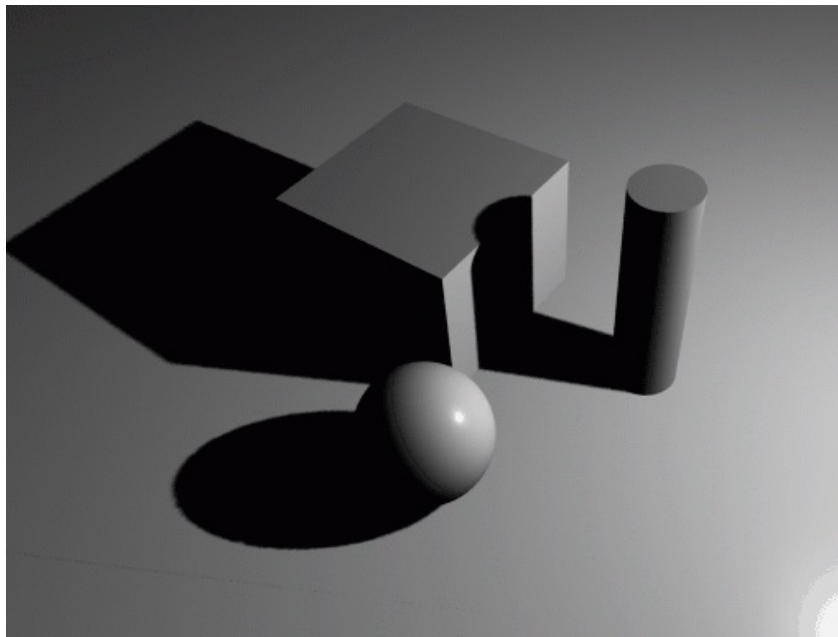
$$\mathbf{I} = \mathbf{I}_a K_a \mathbf{O}_d + \sum_{l=1}^L S_l f_{att,l} \mathbf{I}_{p,l} [K_d \mathbf{O}_d (\mathbf{N} \cdot \mathbf{L}) + K_s \mathbf{O}_s (\mathbf{N} \cdot \mathbf{H})^n]$$

$$S_l = \begin{cases} 0, & \text{if light is blocked} \\ 1, & \text{if light is not blocked} \end{cases}$$

- All or nothing doesn't take into account indirect illumination (light reflected off other surfaces)
- For area light sources, shadow intensity depends on the amount of light that is blocked...

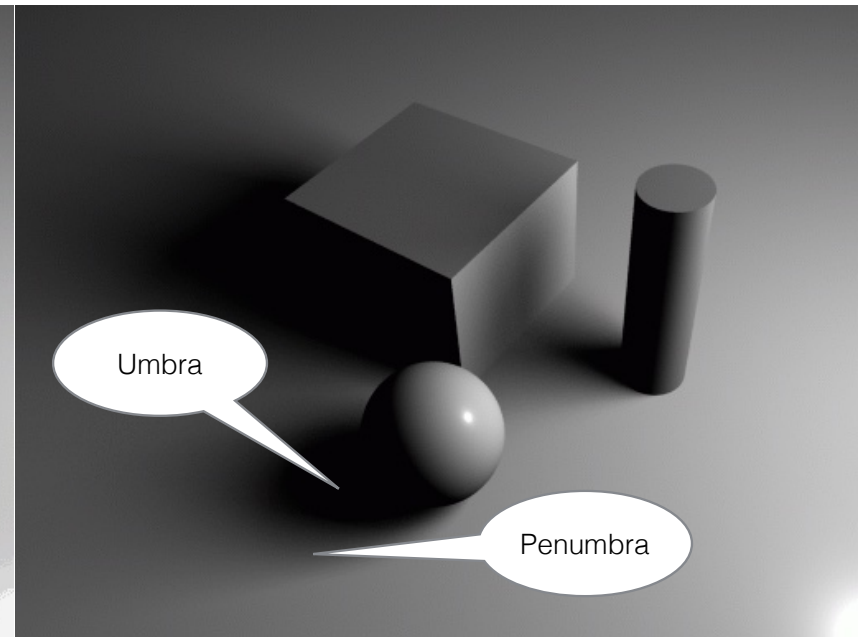
Shadows

Point light sources



hard shadows

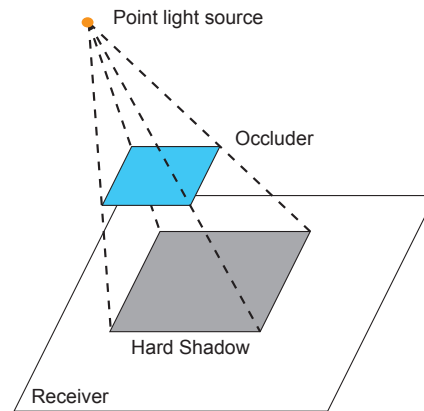
Area light sources



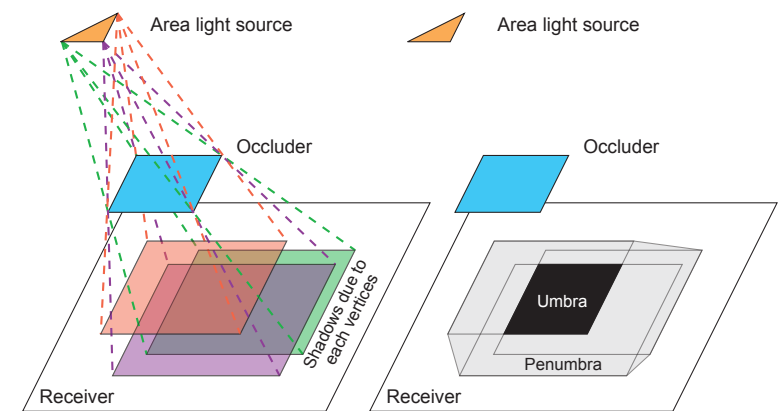
soft shadows

Shadows

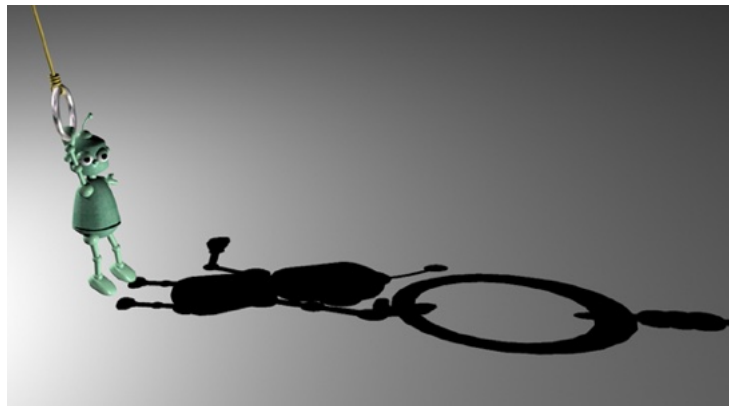
Hard vs Soft Shadows



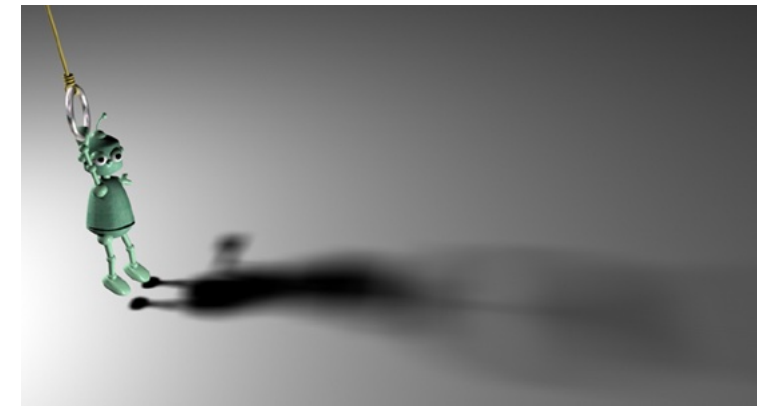
(a) Geometry of hard shadows



(b) Geometry of soft shadows



(c) Illustration of hard shadows

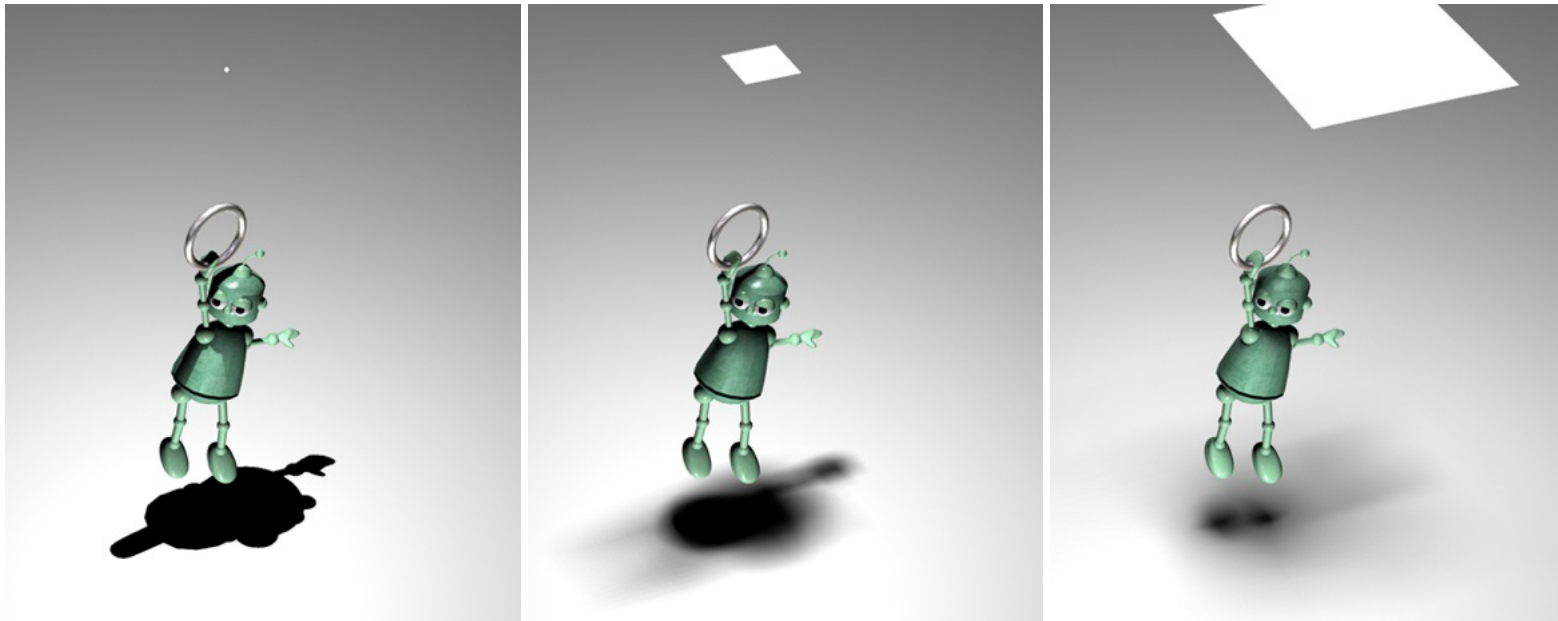


(d) Illustration of soft shadows

Source: <http://maverick.inria.fr/Publications/2003/HLHS03a/SurveyRTSoftShadows.pdf>

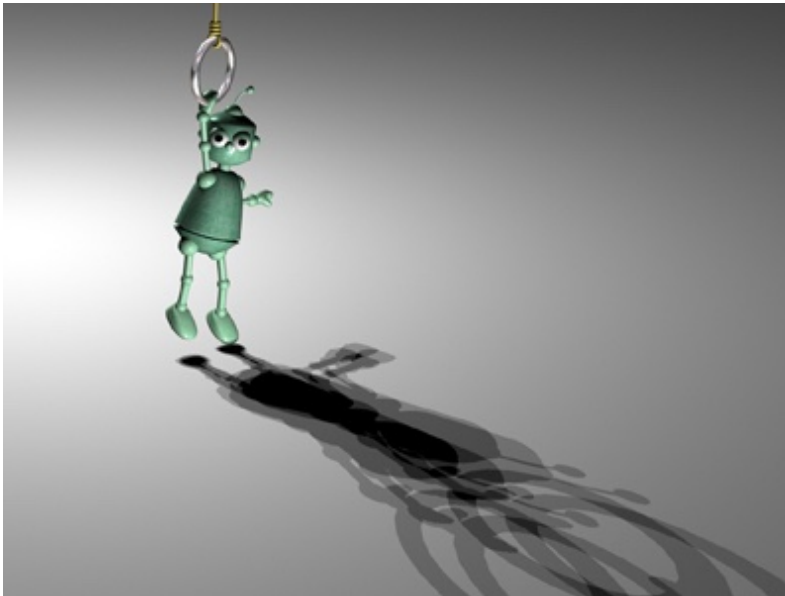
Shadows

Variation of shadow with light source geometry



Shadows

Soft shadows by accumulation of hard shadows



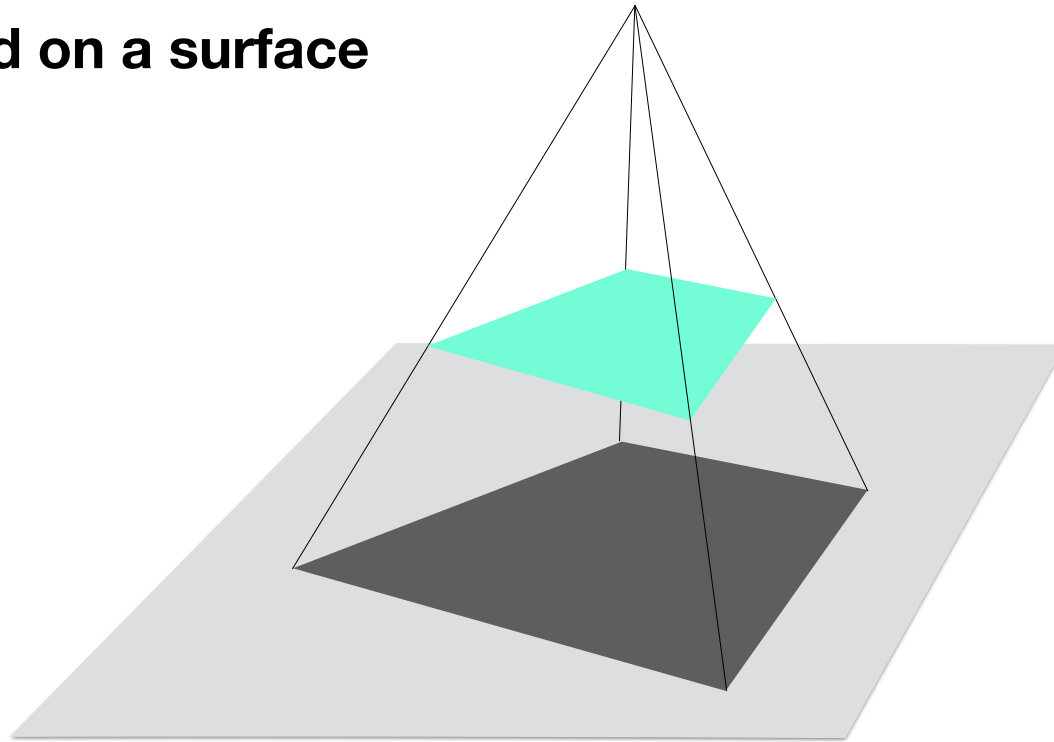
4 samples



1024 samples

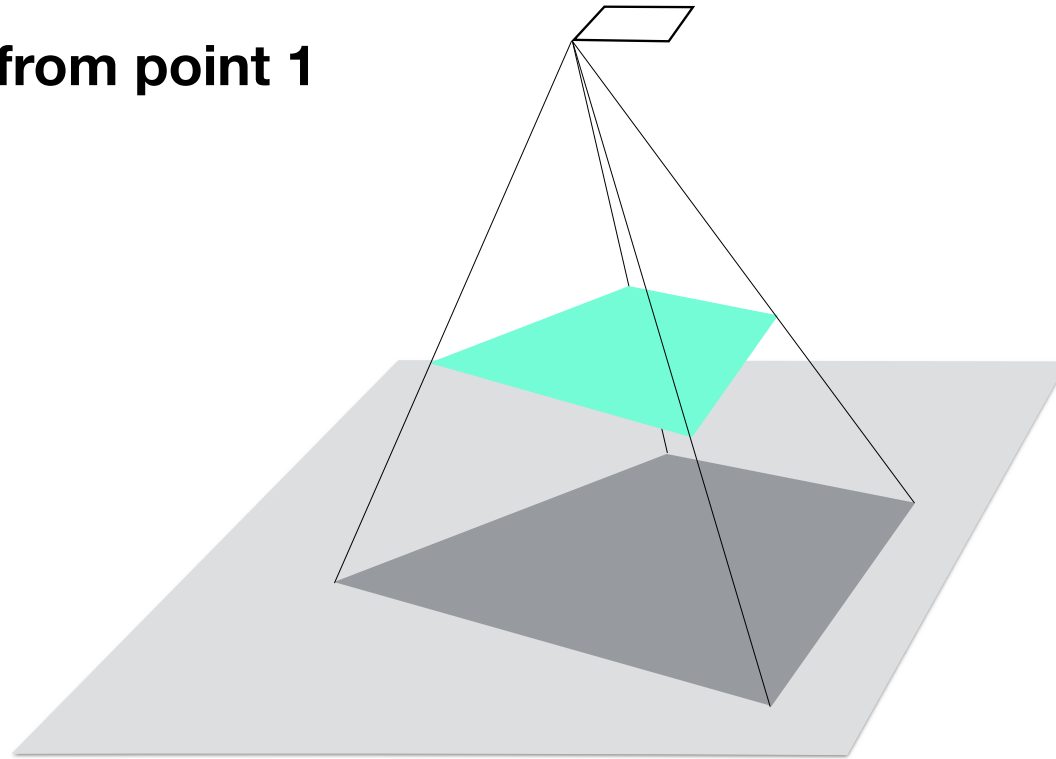
Shadows

A hard shadow projected on a surface



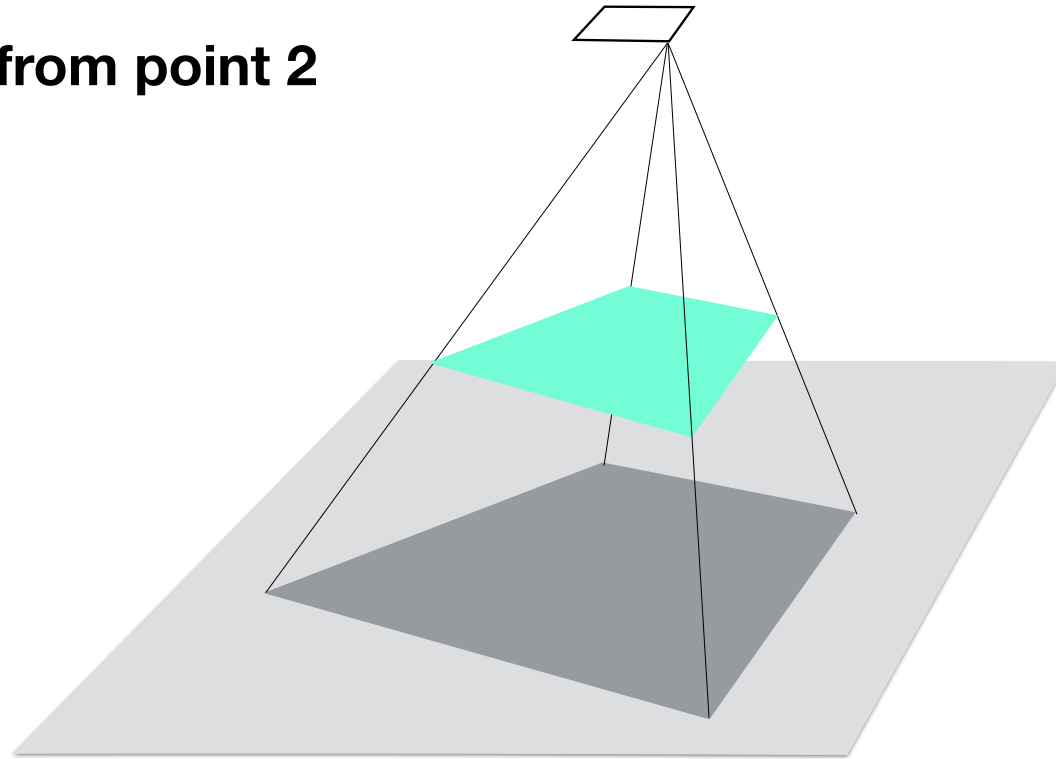
Shadows

Hard shadow produced from point 1



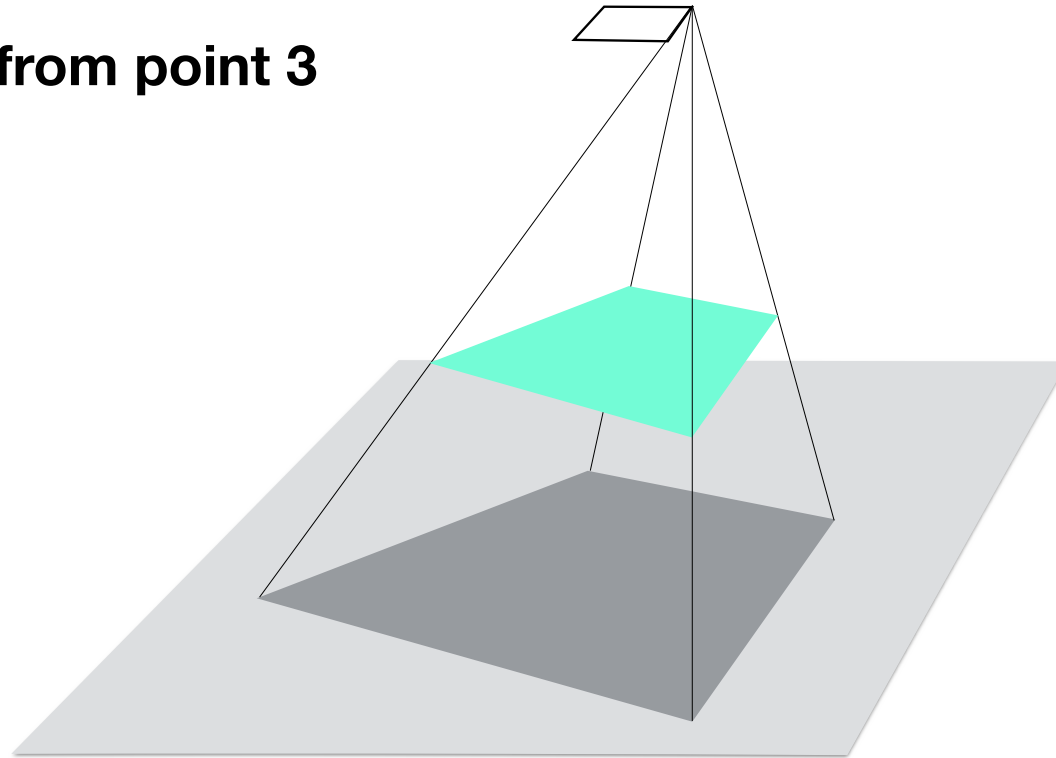
Shadows

Hard shadow produced from point 2



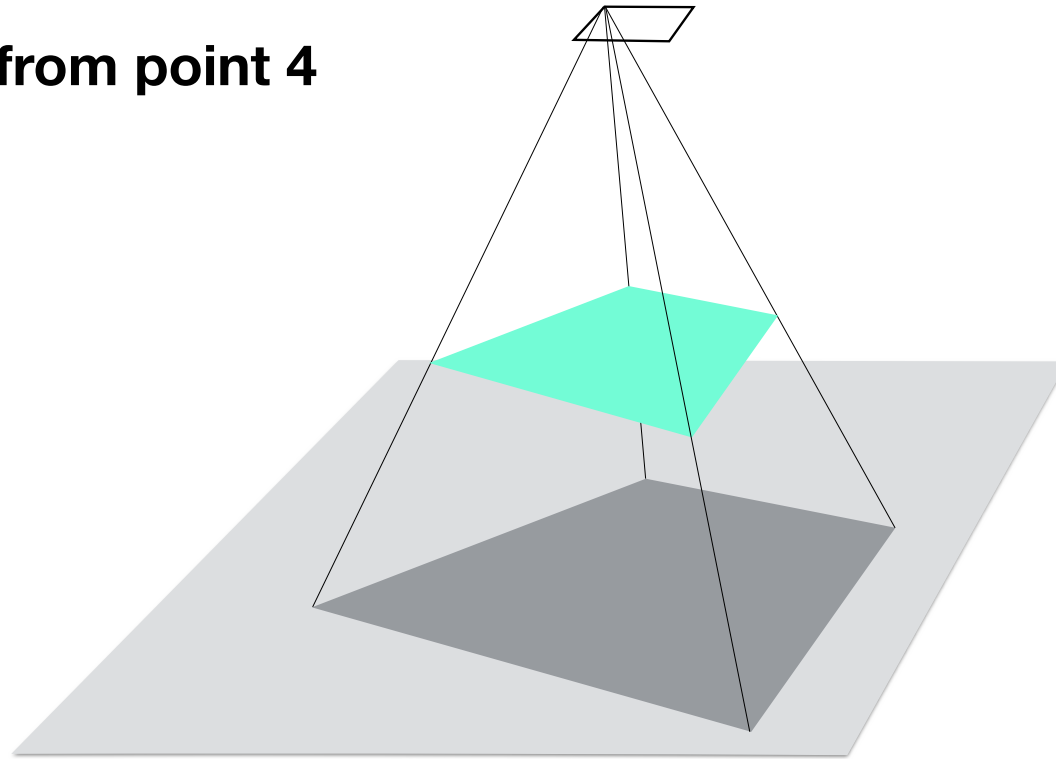
Shadows

Hard shadow produced from point 3



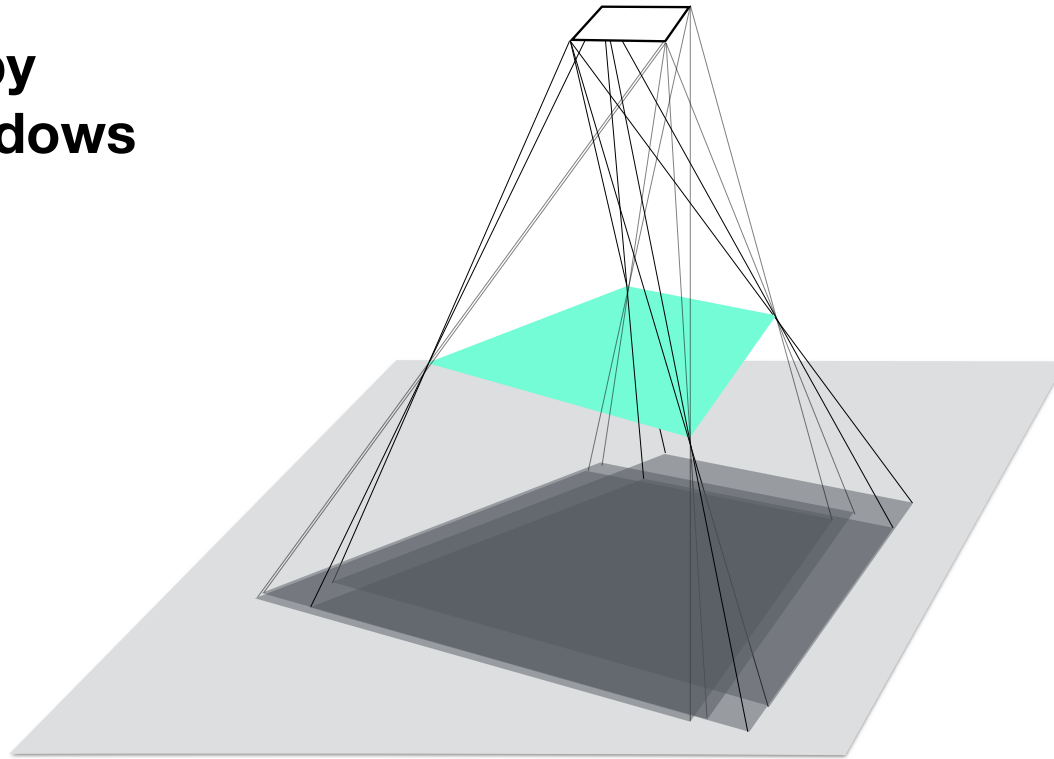
Shadows

Hard shadow produced from point 4



Shadows

**Soft shadow produced by
4 accumulated hard shadows**



Shadows

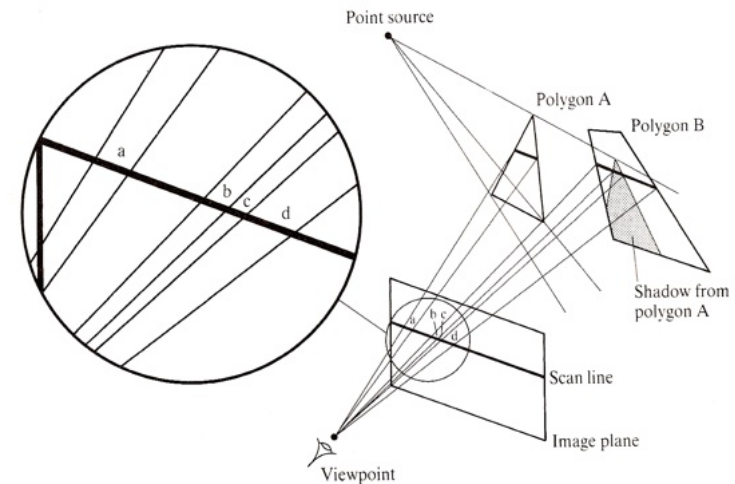
Shadow generation techniques

- 1968-1970: Integration of shadow polygons with a scan-line hidden surface removal algorithm. Shadows are computed while producing the image
- 1978: Shadow generation based on transformations and clipping. Shadows are precomputed and stored in a database
- 1977: 2 Stage generation with shadow volumes. A shadow volume is generated for each light source and accessed during rendering.
- Shadow determination using z-buffer
- Shadow determination with recursive ray-tracing
- Shadow determination with radiosity method

Shadows

Shadows in scan-line algorithms (68-70)

- In a preprocessing step determine for each polygon **all the other polygons** that can possibly shadow it (shadow polygons). This can be done by surrounding the light with a sphere and projecting all polygons onto its surface.
- During scan-line hidden surface removal, for a polygon, if no shadow polygons exist do as normal, otherwise:
 1. shadow polygon completely overlaps current scan-line segment reduce intensity
 2. shadow polygon does not overlap the current scan-line segment shade as normal
 3. shadow polygon partially overlaps current scan-line segment subdivide segment where scan-line segment intersects shadow polygon and restart the step

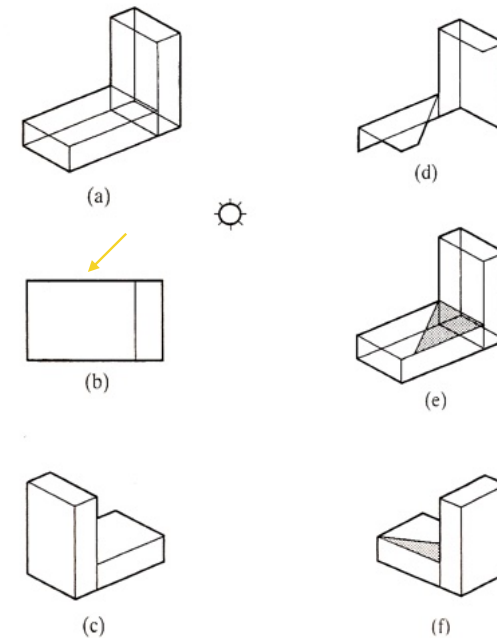


- Limited to objects modelled with polygons
- Limited to point light sources (hard shadows only)
- Preprocessing stage can take a lot of time

Shadows

Shadows with transformations and clipping (78)

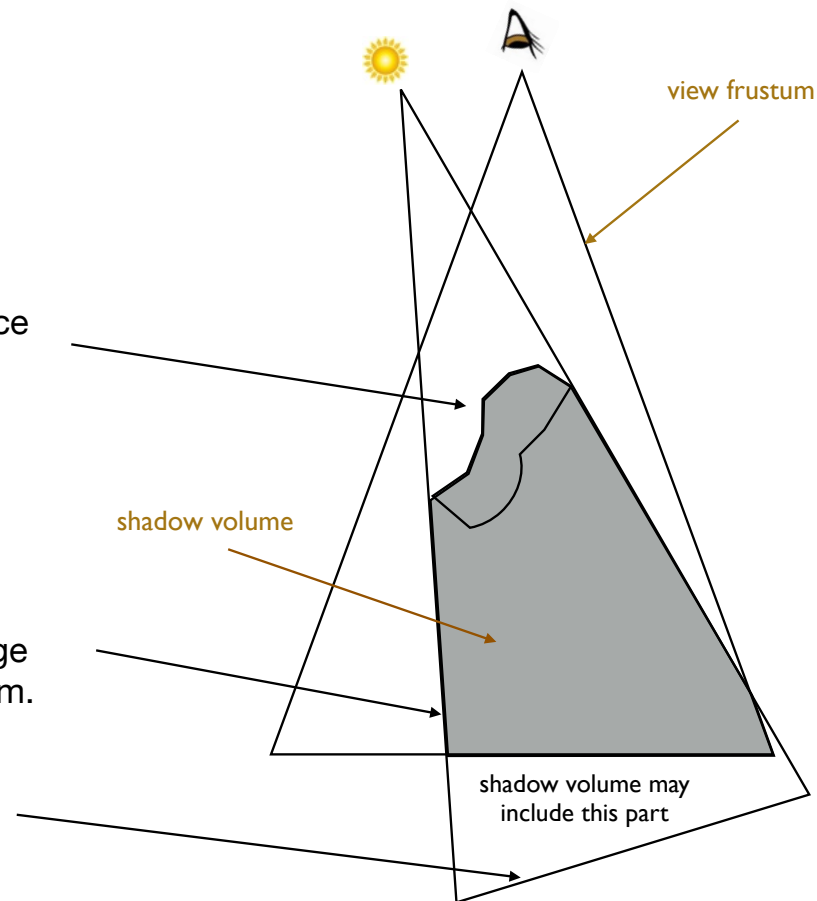
- A Hidden surface removal algorithm (Weiler-Atherton) is applied to a view computed from the light source to find complete surface detail shadow polygons (c-d) by clipping the original surfaces against the lighted parts.
- Shadow polygons from previous step (d) are transformed back to original object coordinates and a new view independent model is created (e).
- Hidden surface removal computed from the viewpoint is applied to the previous model (f)
- Limited to objects modelled with polygons
- hard shadows only
- + Computation is performed with object resolution (exact)
- + Illuminated/Shadowed areas can be computed (CAD)



Shadows

Shadows with Shadow Volumes (77)

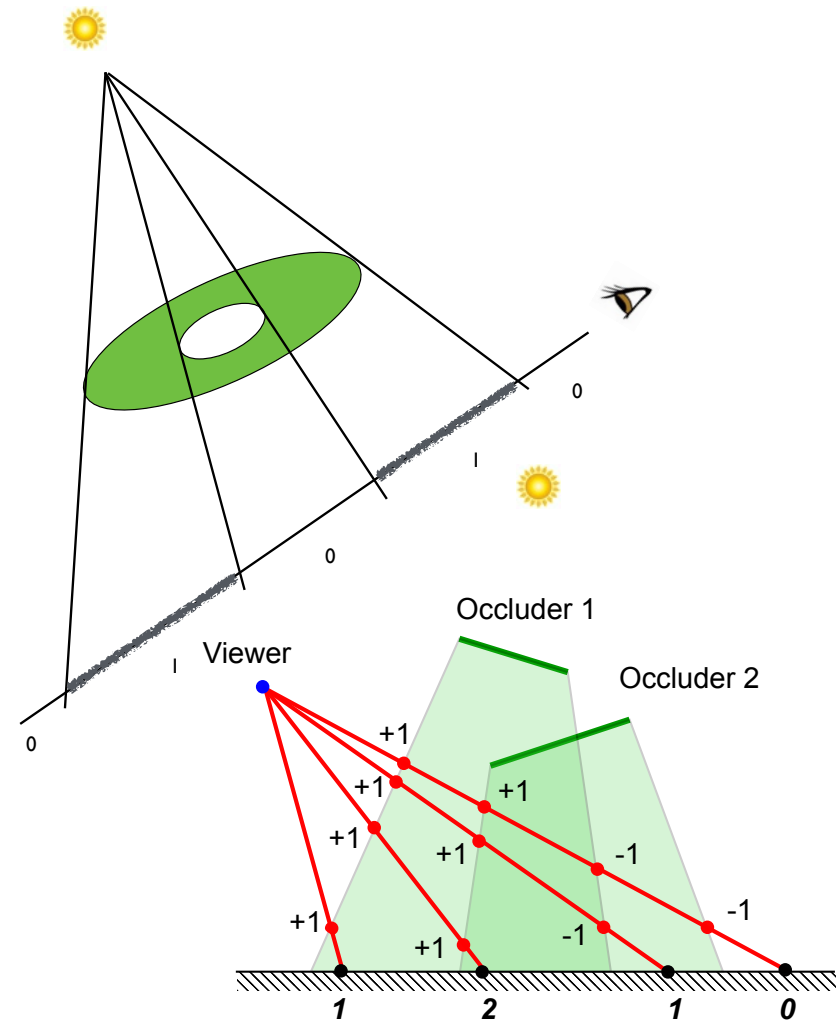
- Shadow volumes are defined for polygonal objects that may cast shadows
- The shadow volume determines the portion of space in the shadow of an object for a particular point light source
- The shadow volume is defined by the front facing polygons of the object viewed from the light source...
- ... quadrilaterals with one edge being a silhouette edge of the object...
- ... 2 edges connecting the vertices of the silhouette edge to points far away enough to be outside the view frustum. These two edges, if prolonged, would pass through the point light
- ... and a remaining edge outside the view volume



Shadows

Shadow Volumes in use

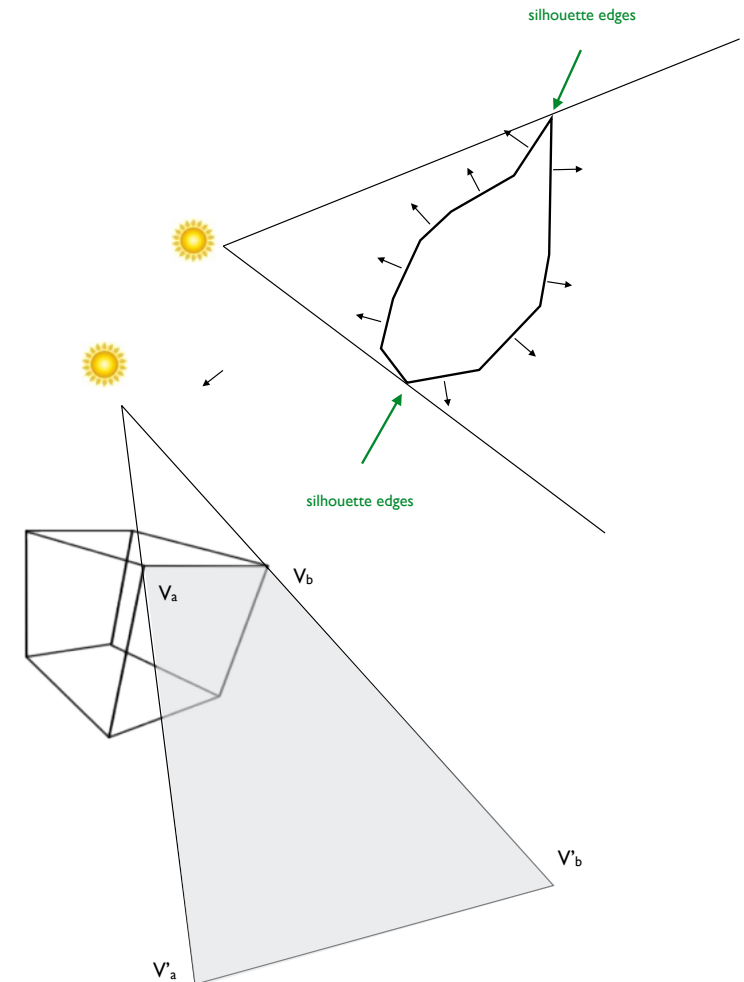
- The basic idea is that if we sort the shadow volume polygons in increasing order of depth from the viewpoint...
- ... and distinguish between front facing and back facing shadow polygons...
- ...crossing a front facing shadow polygon will increase the shadow counter (entering the shadow volume)...
- ... crossing a back facing shadow polygon will decrease the shadow counter (leaving the shadow volume)...
- The shadow counter starts with:
 - 0 if viewpoint is outside the shadow volume
 - 1 if viewpoint is inside the shadow volume



Shadows

Shadow Volumes computation

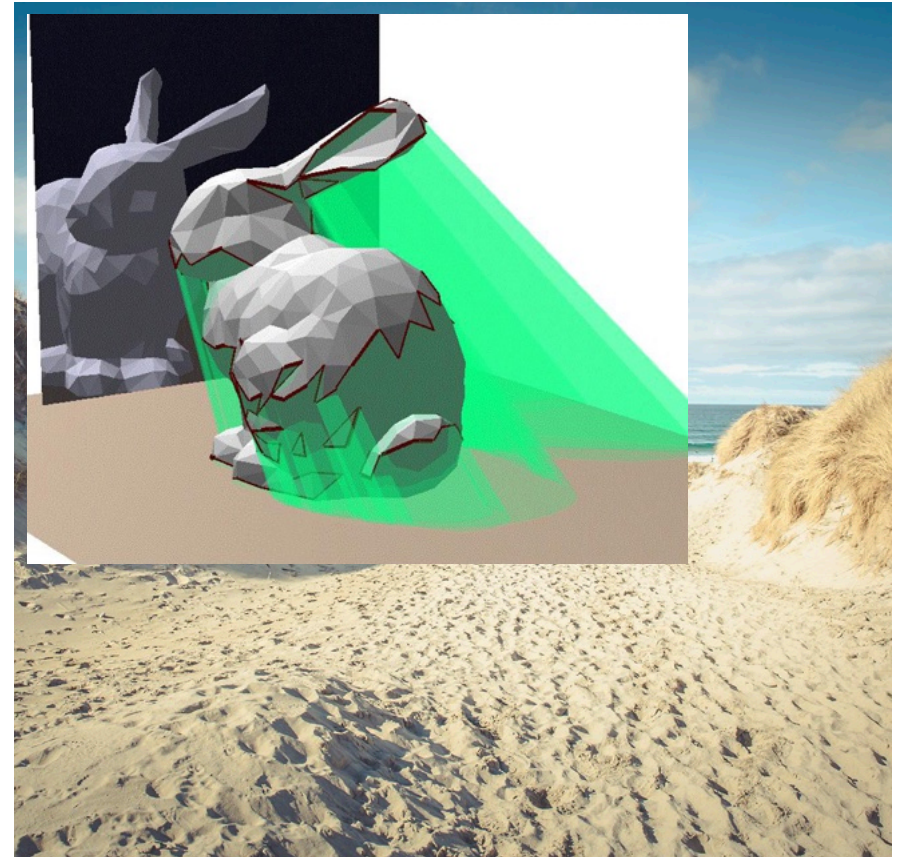
- All front facing polygons (from the light point of view) are shadow volume polygons (the front cap)
- To compute the side walls of the shadow volume we need to determine which edges of the B-rep are silhouette edges.
- A silhouette edge is an edge connecting two faces of the model such that:
 - the normal of one of the faces points towards the light source
 - the normal of the other face points away from the light source
- A boundary edge is also a silhouette edge
- If the vertices of a silhouette edge are V_a and V_b , we extrude the edge away from the light. Determine two additional points V'_a and V'_b , outside the view volume such that L (the light position), V_a and V'_a are co-linear, as well as L , V_b and V'_b
- Finally connect all V'^* points to form the other cap of the volume



Shadows

Stenciled Shadow Volumes (91)

1. Render the scene as if it were completely in shadow
 2. For each light source construct a mask in the stencil buffer with holes where the visible surface is not inside a shadow volume
 3. Render the scene again as if it were completely lit. Stenciled pixels will get this render results added to the shadowed rendering of the first pass.
- There are 3 variations on how to produce the mask in step 2: depth-pass, depth-fail and exclusive-or



Shadows

Stenciled Shadow Volumes: Depth pass*

1. Disable z-buffer and colour buffer
 2. Turn back-face culling on
 3. Set stencil operation to increment on depth pass (count shadow volume polygons in front of the object)
 4. Render shadow volumes (only front faces will be drawn)
 5. Turn front-face culling on
 6. Set stencil operation to decrement on depth pass
 7. Render shadow volumes (only back faces will be draw)
- + Shadow volume doesn't need to be capped at the rear end
- Doesn't work if viewpoint is inside shadow volume (light behind object or object casting shadow behind the viewpoint)

* algorithm starts with an unlit version of the model already rendered, including the depth buffer values

Shadows

Stenciled Shadow Volumes: Depth fail*

1. Disable z-buffer and colour buffer
 2. Front-face culling on
 3. Set stencil to increment on depth fail (count shadows behind the object)
 4. Render shadow volumes
 5. Back-face culling on
 6. Set stencil operation to decrement on depth fail
 7. Render shadow volumes
- + Works for viewpoints inside shadow volumes too because shadow surfaces between eye and object are not counted
 - Shadow volume needs to be capped at the rear

* algorithm starts with an unlit version of the model already rendered, including the depth buffer values

Shadows

Stenciled Shadow Volumes: Exclusive or*

1. Disable z-buffer and colour buffer
2. Set stencil operation to XOR on depth pass
3. Render shadow volumes

+ Saves a rendering step

- Does not work for intersecting shadow volumes

* algorithm starts with an unlit version of the model already rendered, including the depth buffer values

Shadows

2 pass shadow generation with z-buffer (image precision)

1. Generate a view from the light and keep the z-buffer result
2. Use a modified z-buffer algorithm to compute the image from the viewer:
 1. For every visible pixel, transform to light coordinate system (x', y', z') and use (x', y') to access the light z-buffer value z_L .
 2. If z_L is nearer the camera than z' then there is some other object blocking the light and the pixel is in shadow. Otherwise it is lit.

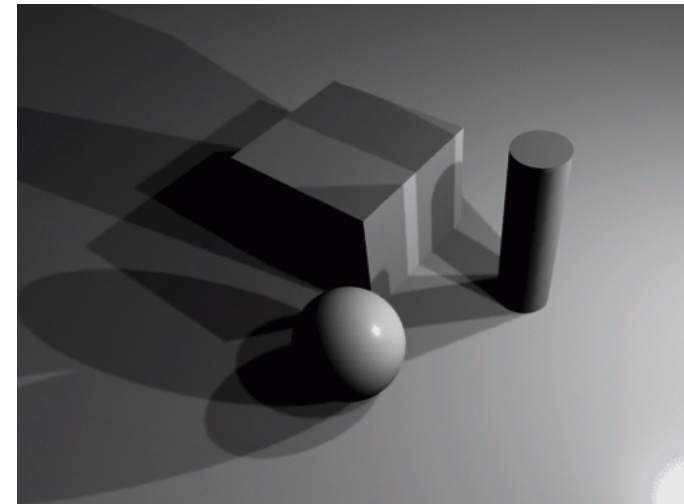
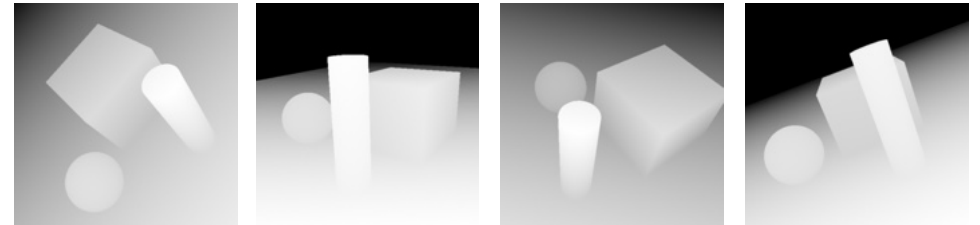


Shadow map

Shadows

2 pass shadow generation with z-buffer (image precision)

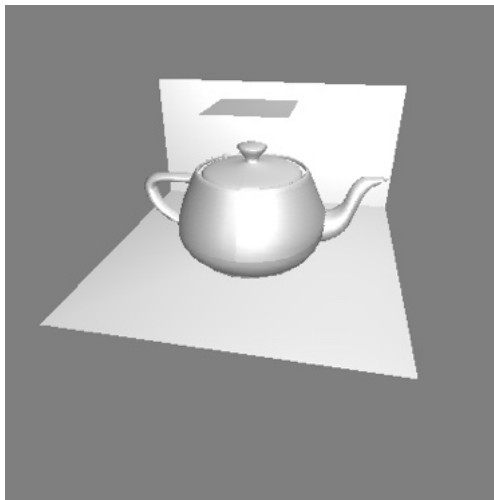
- Several light sources can be handled. For each light source generate a shadow map (z-buffer from light's point of view)
- More adequate for spotlights or lights at infinity
- + Hardware accelerated
- Aliasing artefacts (shadow acne)



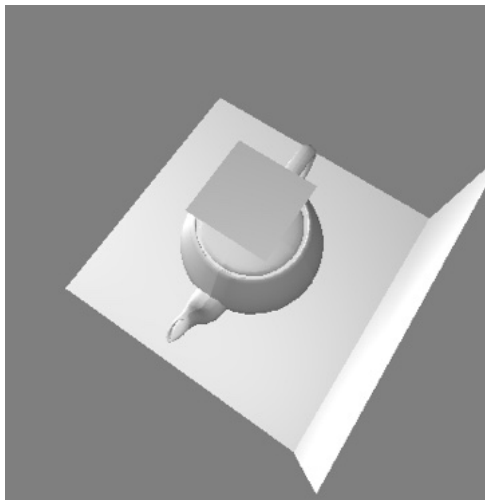
Multiple shadows

Shadows

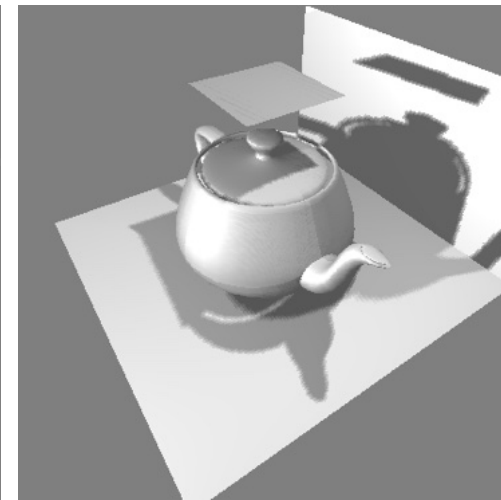
2 pass shadow generation with z-buffer (image precision)



View from light 1



View from light 2

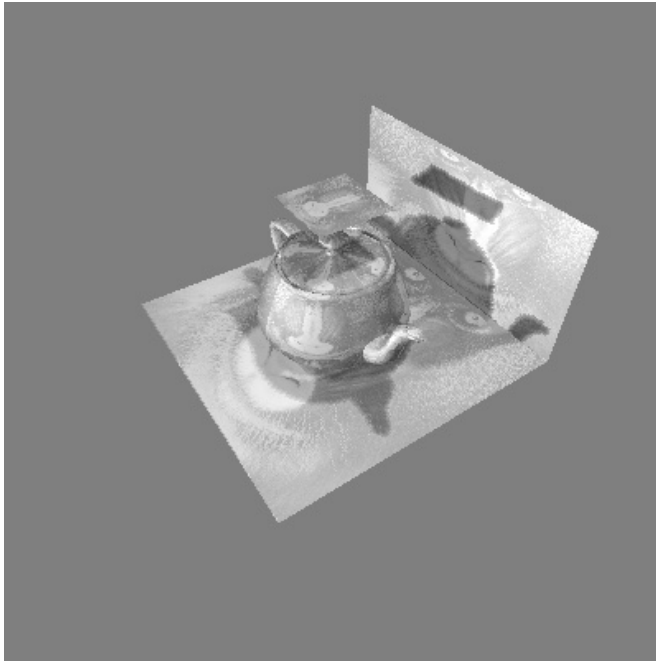


View from viewer

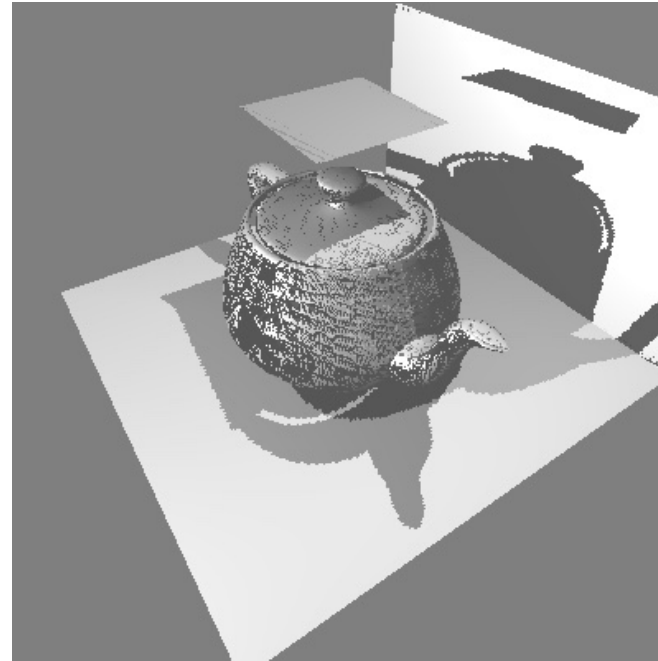
Example from <http://web.cs.wpi.edu/~matt/courses/cs563/talks/shadow/shadow.html>

Shadows

2 pass shadow generation with z-buffer (image precision)



Shadows on texture mapped objects



Wrong shadows from precision problems

Mapping Applications

Property	Map	Technique
kd, diffuse reflectivity	UV	Diffuse detail mapping, like the upholstery on a sofa
ks, glossy reflectivity	UV	Glossy detail, like the part of a tarnished doorknob that's polished by constant use
Lin	Reflection	Environment mapping
Lout	UV	Light mapping. Texture mapping is used to specify the emissivity of an object like a neon lamp
Position or normal vector	UV	Bump mapping or displacement mapping
Visibility of a light source	Perspective projection	Shadow mapping
Artistic Lout	Various dot products	XToon shading in expressive rendering